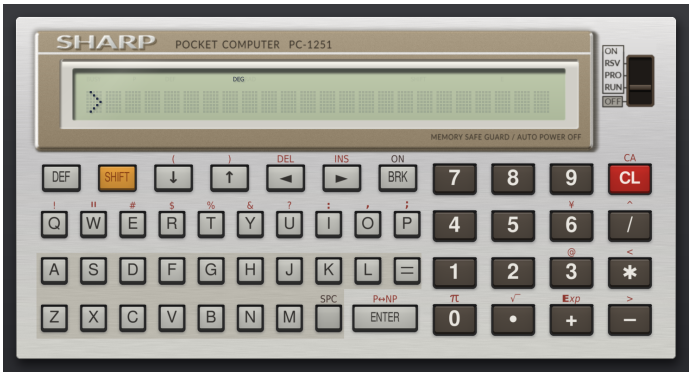
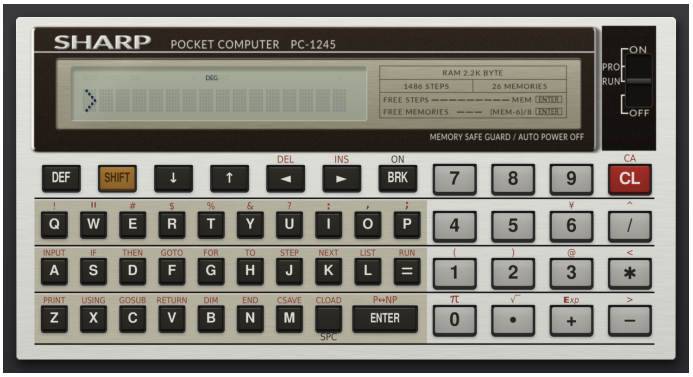
SHARP PC-1251/PC-1245/PC-1261/PC-1350 エミュレータ

使い方の手引き





おりぐち

エミュレータ 0.7.2 版

ROM は付属していません。ROM がなくても、BASIC が動き、マシン語のプログラミングもできます (6 節)。

目次

1 はじめに 3

2 インストールと起動 4
2.1 端末を開く . . . 4
2.2 uv をインストールする . . . 4
2.3 エミュレータのフォルダを用意する . . . 4
2.4 ROM を用意する (ROM で動かすとき) . . . 5
2.5 起動する . . . 6
2.6 終わる . . . 7
2.7 うまく動かないとき . . . 7

3 画面と操作 8
3.1 各部の名前 . . . 8
3.2 マウスで操作する . . . 9
3.3 パソコンのキーボードで操作する . . . 9
3.4 エミュレータの操作 . . . 9
3.5 ゲームパッドで操作する . . . 12
3.6 状態を保存する (どこでもセーブ) . . . 13
3.7 画面と音を録画する . . . 13
3.8 モードスイッチ . . . 14
3.9 SHIFT と DEF . . . 14

4 使ってみる 14
4.1 電卓として使う . . . 14
4.2 BASIC のプログラムを打ち込んで動かす . . . 14
4.3 打ち込みを楽にする、行を直す・消す . . . 15
4.4 PRINT で止めずに見せる (WAIT) . . . 16
4.5 DEF キーで実行する . . . 17
4.6 RSV モードで文字列を登録する . . . 18
4.7 エラーが出たら . . . 18
4.8 電源とメモリ . . . 18

5 プログラムのファイルを使う 19
5.1 プログラムのフォルダに置いて、一覧から読み込む . . . 19
5.2 ファイルの形式 . . . 21
5.3 端末から読み込む . . . 24
5.4 マシン語を動かす . . . 24
5.5 CALL で表示をつけたまま計算する . . . 24
5.6 マシン語で液晶に描く . . . 25
5.7 PC-インタープリタともぐらたたき . . . 26
5.8 ほかの読み込み方 . . . 27
5.9 プログラムをファイルに書き出す . . . 27
5.10 プリンタに印字する . . . 28
5.11 カセットの音 (wav) で実機とやり取りする . . . 29

6 ROM なしで使う 30
6.1 ROM 使用時と違うところ . . . 30
6.2 代替 ROM ルーチンを記述する . . . 31
6.3 作業欄だけで動かす . . . 32

7 マシン語の作業欄 32
7.1 作業欄の各部 . . . 33
7.2 BASIC の CALL で呼んで調べる . . . 33
7.3 ダンプリストの読み方 . . . 34
7.4 ダンプを打ち込む . . . 34
7.5 実行して調べる . . . 36
7.6 命令とデータを見分ける . . . 37
7.7 打ち込んだマシン語をファイルに書き出す . . . 37

8 PC-1250・PC-1255 として使う 38

9 PC-1245 として使う 39
9.1 液晶 . . . 39
9.2 モードスイッチ . . . 39
9.3 キー . . . 39
9.4 メモリと見本のプログラム . . . 40

10 PC-1261 として使う 40
10.1 液晶 . . . 41
10.2 キー . . . 41
10.3 メモリと見本のプログラム . . . 41

11 PC-1350 として使う 42
11.1 電源とモード . . . 42
11.2 液晶 . . . 43
11.3 キー . . . 43
11.4 グラフィック . . . 43
11.5 シリアル入出力 . . . 43
11.6 TEXT モードと BASIC モード . . . 44
11.7 メモリと見本のプログラム . . . 44

付録 A 起動のオプション 44

付録 B SHIFT で入る記号 45

付録 C メモリマップと入出力 46

付録 D 実機と違うところ 47

付録 E ファイルの一覧 48

付録 F 謝辞 48

1　はじめに

sc61box は、SHARP のポケコンに使われた CPU SC61860 のエミュレータです。PC-1251、PC-1245、PC-1261、PC-1350 と、RAM の大きさだけが違う PC-1250・PC-1255、PC-1260・PC-1262 として動作します。キーとモードスイッチはマウスでも操作できます。

ROM はシャープの著作物のため、エミュレータには含めていません。ROM がなくても、どの機種もエミュレータに内蔵した BASIC(ROM なしの BASIC) で動作し、BASIC のプログラムを入力して実行できます。マシン語も、BASIC の `POKE` と `CALL`、または 1 命令ずつ確認できる作業欄 (7 節) で入力して実行できます。ROM なしの動作は ROM 使用時とまったく同じではありません。違いは 6 節にまとめています。実機から読み出した ROM を置けば (2.4 節)、内部 ROM と BASIC ROM を命令単位で実行します。

電卓としても使えます。BASIC のテキストや 16 進ダンプのファイルは、一覧から選んで読み込めます。終了してもプログラムは消えません。

どの機種にするかは起動のときに選び、何も指定しなければ PC-1251 になります。この手引きは PC-1251 を例に書き、ほかの機種で違うところは機種ごとの節 (PC-1250・PC-1255 は 8 節、PC-1245 は 9 節、PC-1261 は 10 節、PC-1350 は 11 節) にまとめました。

SHARP はシャープ株式会社の商標です。このエミュレータは個人が作った非公式のもので、シャープ株式会社とは関係ありません。同梱した見本のプログラムは、PC-インタープリタともぐらたたき、LC-3 のコンパイラと記事の見本を除き、このエミュレータのために書いたものです。

動作環境

- macOS、Windows(10 以降)、Linux。
- Python 3.10 以降 (次の節でインストールする uv が用意します)。
- 初回の起動のときだけ、インターネットへの接続が要ります。

キーとコマンドの書き方

パソコンのキーは Esc、本体のキーは SHIFT のように書き分けます。command/Ctrl+O は、macOS では command を、Windows と Linux では Ctrl を押しながら O を押すという意味です。macOS と Windows で違うところは「macOS」「Windows」と見出しをつけて両方を書きました。端末 (macOS の「ターミナル」、Windows の「PowerShell」) で打つコマンドは、次のような枠に入れてあります。行の頭の$や>は打たなくてかまいません。

```
$ uv run sc61box
```

2　インストールと起動

2.1　端末を開く

■macOS　「アプリケーション」→「ユーティリティ」→「ターミナル」を開きます。Spotlight(command+Space) で「ターミナル」と打っても開けます。

■Windows　スタートメニューで「PowerShell」と打ち、「Windows PowerShell」を開きます。「ターミナル」(Windows Terminal) が入っていれば、「ターミナル」を使ってもかまいません。

2.2　uv をインストールする

uv は Python とライブラリをまとめて扱う道具です。

■macOS

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

■Windows

```
> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

インストールが終わったら、端末をいったん閉じて開き直します。次のコマンドでバージョン番号が出れば成功です。

```
$ uv --version
```

Homebrew や winget を使っている場合

macOS で `brew install uv`、Windows で `winget install --id=astral-sh.uv -e` としても同じ uv が入ります。

2.3　エミュレータのフォルダを用意する

エミュレータは GitHub のページ (https://github.com/woriguchi/sc61box) にあります。git があれば、置きたい場所で次のコマンドを打つと、`sc61box` フォルダに取ってきます。

```
$ git clone https://github.com/woriguchi/sc61box.git
```

git がなければ、GitHub のページの「Code」→「Download ZIP」でダウンロードした ZIP ファイルを展開して、好きな場所に置きます。展開したフォルダの名前は `sc61box-main` になるので、

sc61box に変えておきます (以下では sc61box と書きます)。端末で sc61box に移ってからコマンドを打ちます。

前の版から新しくする場合

0.7.0 で、GitHub の履歴を書き直しました。そのため、0.6.1 までに git clone で取ってきたフォルダは、git pull では新しい版にできません (「divergent branches」と出ます)。フォルダに移ってから git fetch --force --tags と git reset --hard origin/main を続けて打つと、新しい版に合わせられます。rom フォルダと、ホームの.sc61box フォルダ (保存した RAM やプログラム) は、そのまま残ります。エミュレータのフォルダの中で書き換えたファイルは、元に戻ります。そのあとは、git pull で新しい版にできます。
前の名前 (pc1251-emulator) で取ってきたフォルダも、同じようにして新しい版にできます。起動のコマンドは uv run sc61box です。データのフォルダ (ホームの.pc1251) は、初めて起動したときに.sc61box へ移します。前の名前の環境変数 (PC1251_ROM など) も使えます。

■macOS　(ホームの src の下に置いた場合)

```
$ cd ~/src/sc61box
```

■Windows　(「ドキュメント」の下に置いた場合)

```
> cd $HOME\Documents\sc61box
```

フォルダの場所を打つのが面倒なら、cd と空白を打ってから、Finder やエクスプローラーのフォルダをターミナルや PowerShell のウィンドウにドラッグ&ドロップすると、場所が入ります。

2.4　ROM を用意する (ROM で動かすとき)

ROM がなくても起動でき、ROM なしの BASIC で動きます (6 節)。実機と同じ BASIC で動かすときは、実機から読み出した ROM を用意します。

機種ごとに 2 つのファイルが要ります。エミュレータのフォルダの中に rom フォルダを作り、表のファイル名で置きます (置き方の例は表のあと)。使う機種のぶんだけあれば動き、両方を並べて置いてもかまいません。

ファイル名	中身	大きさ
cpu-1251.rom	PC-1251 の CPU(SC61860) の内部 ROM、0000〜1FFF 番地	8192 バイト
bas-1251.rom	PC-1251 の BASIC ROM、4000〜7FFF 番地	16384 バイト
cpu-1245.rom	PC-1245 の CPU の内部 ROM、0000〜1FFF 番地	8192 バイト
bas-1245.rom	PC-1245 の BASIC ROM、4000〜7FFF 番地	16384 バイト
cpu-1260.rom	PC-1260/1261/1262 の CPU の内部 ROM、0000〜1FFF 番地	8192 バイト
bas-1260.rom	PC-1260/1261/1262 の BASIC ROM、8000〜FFFF 番地	32768 バイト
cpu-1350.rom	PC-1350 の CPU の内部 ROM、0000〜1FFF 番地	8192 バイト
bas-1350.rom	PC-1350 の BASIC ROM、8000〜FFFF 番地	32768 バイト

たとえば macOS でホームの src の下に置いた場合は、次のようになります。

```
~/src/sc61box/
    sc61box/
    programs/
    rom/
        cpu-1251.rom
        bas-1251.rom
        cpu-1245.rom      (PC-1245 も使うとき)
        bas-1245.rom
        cpu-1260.rom      (PC-1261 も使うとき)
        bas-1260.rom
```

rom フォルダは、エミュレータのフォルダに移ってから次のコマンドで作れます (macOS でも Windows でも同じです)。Finder やエクスプローラーで ROM のファイルを rom フォルダへ入れます。

```
$ mkdir rom
```

Windows で「ドキュメント」の下に置いた場合は、C:\Users\名前\Documents\sc61box\rom\cpu-1251.rom のようになります。

実機からの読み出し方は、yagshi 氏のページ「Extracting ROM Data (ROM 吸い出し)」(https://www.oit.ac.jp/rd/labs/kobayashi-lab/~yagshi/old_web/misc/pocketcom/extrom.html) に PC-1251 の手順が、ht-deko.com のページ「SHARP PC-1245/50/51/55 の使い方」(https://ht-deko.com/pc1245/) に PC-1245 の手順があります。

ファイルが見つからないときや大きさが足りないときは、ROM なしで起動します。どの機種も ROM なしの BASIC で動きます (6 節)。どれで動いているかは、ウィンドウの名前の終わりに「(ROM BASIC)」「(ROM-less BASIC)」「(no ROM, workbench only)」と出ます。

初めて uv run を使うとき

最初の uv run では、uv が Python とライブラリをダウンロードするので、しばらく時間がかかります。2 回目からはすぐに始まります。

ROM を rom 以外のフォルダに置くときは、環境変数 SC61BOX_ROM にフォルダの場所を入れておきます。ほかに、ホームの.sc61box フォルダも探します。

2.5　起動する

```
$ uv run sc61box
```

PC-1251 の画面がウィンドウに出て、液晶に>(プロンプト) が出れば動いています。最初の 8 秒ほ

どは、画面の中ほどにキーの説明 (パソコンのキーとポケコンのキーで対応が違うもの) が出ます。何かキーを押すかクリックすると消えます (command / Ctrl + / でいつでも出し入れできます)。

ほかの機種で起動するときは、`--model 1245` のように機種の番号を付けます (付録付録 A)。

画面が大きすぎたり小さすぎたりするときは、拡大率を指定します。画面に収まらないときは自動で縮めます。

```
$ uv run sc61box --scale 0.8
```

画面の文は英語にもできます。操作欄の「表示・入力」のメニューで「English」を選ぶと英語になり、次に起動したときも英語です (英語の画面で「日本語」を選ぶと戻ります)。起動のときの`--lang en`・`--lang ja` や、環境変数 `SC61BOX_LANG` でも選べます。どれもなければ、パソコンの言語の設定に合わせます。

```
$ uv run sc61box --lang en
```

2.6　終わる

ウィンドウを閉じれば終わります。終わるときに RAM の中身を保存するので (4.8 節)、次に起動すると、プログラムも変数もそのまま残っています。保存した RAM を読み込んだときは、起動時に画面の下に「前回終了時の RAM を読み込みました」と表示します。

保存したものを使わずにまっさらな状態で始めたいときは`--fresh` をつけます。`--fresh` をつけたときは、終わるときにも保存しません。

```
$ uv run sc61box --fresh
```

2.7　うまく動かないとき

ウィンドウの名前の終わりに「(ROM-less BASIC)」か「(no ROM, workbench only)」と出る

ROM のファイルが見つからないか、大きさが足りないので、ROM なしで起動しています (6 節)。ROM で動かすときは、2.4 節を見て `rom` フォルダに置いてください。

uv というコマンドがないと言われる

uv を入れたあと、端末を開き直していません。閉じて開き直してください。

Windows で「スクリプトの実行が無効」と出る

uv をインストールするコマンドを、上に書いたとおり`-ExecutionPolicy ByPass` をつけて打ってください。

キーを打っても字が入らない

日本語入力になっていると、打ったキーがエミュレータに伝わりません。英数入力に切り替えてく

ださい。

Mac で F1〜F12 を押しても反応しない

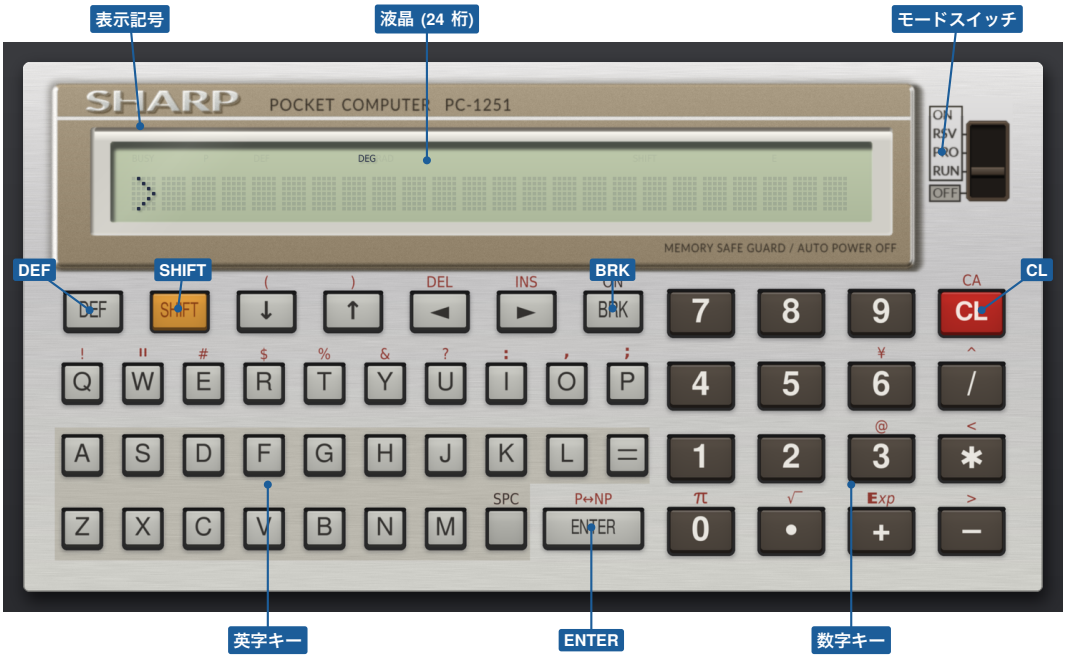
ノート型の Mac では、F1〜F12 は明るさや音量のキーになっていることがあります。fn を押しながら押すか、3.4 節の command / Ctrl + O などの組み合わせか、本体の下の操作欄のボタンを使ってください。

音がいらない

音を止めたいときは`--mute`をつけて起動します。

3　画面と操作

3.1　各部の名前



液晶

5×7 ドットの文字が 24 桁並びます。

表示記号

液晶の上の小さな字です。`DEG`(角度の単位)、`SHIFT`(SHIFT を押した)、`DEF`、`BUSY`(計算中・実行中)、`E`(エラー) などが出ます。いまのモードは液晶には出ないので、スイッチの位置で見ます。

モードスイッチ

右上の縦のスイッチです。上から `RSV`、`PRO`、`RUN`、`OFF`。いちばん上の `ON` はスイッチの位置ではなく、BRK キーが ON(電源を入れる) を兼ねるという表示です。

3.2　マウスで操作する

- 画面のキーをクリックすると、本体の同じキーを押したことになります。ボタンを押しているあいだはキーも押しっぱなしです。
- モードスイッチは、ラベル (RSV〜OFF) かつまみのあたりをクリックすると、クリックした位置に動きます。つまみを上下にドラッグしても動かせます。

3.3　パソコンのキーボードで操作する

英字・数字・Enter・Space・矢印キーは、PC-1251 の同じキーとして働きます。押しているあいだは押しっぱなしになります。記号は、パソコンで打った字を PC-1251 での打ち方 (SHIFT のあとにキー) に直して打ち込みます。たとえば:を打つと、SHIFT I と押したことになります。

ROM が入力を待っているあいだは、速く打ったキーも取りこぼさないように、1 つずつ順に届けます。プログラムが動いているあいだ (INKEY$やマシン語でキーを読むゲームなど) は、パソコンのキーを押しているあいだ本体のキーを押します。いくつかのキーを同時に押すこともできます (DEF を押したまま 6 を押すなど)。

macOS	Windows	PC-1251	説明
英数字・Enter・矢印	同じ	同じキー	押しているあいだは押しっぱなし
Space	Space	SPC	
記号	記号	SHIFT+ キー	打った字に合わせて直す (付録 B)
Tab	Tab	SHIFT	
Option	Alt	DEF	
Delete (fn + delete)	Delete	CL	表示を消す
delete	Backspace	◀ と DEL	1 字戻って消す
Esc	Esc	BRK	実行を止める。電源が切れているときは ON
\	\	SHIFT 6	¥を打つ (ポケコンにはバックスラッシュがない)

3.4　エミュレータの操作

モードスイッチの切り替えや一覧の表示など、PC-1251 のキーにないエミュレータの操作は、command (Windows と Linux では Ctrl) との組み合わせで行います。

macOS	Windows	すること
command + ↑ ↓	Ctrl + ↑ ↓	モードスイッチを 1 段上・下へ動かす
command + O	Ctrl + O	プログラムの一覧 (5 節)
command + R	Ctrl + R	裏の RESET ボタンを押す
command + T	Ctrl + T	速くする (1/2 倍、1 倍、2 倍、4 倍、8 倍)
shift + command + T	Shift + Ctrl + T	遅くする

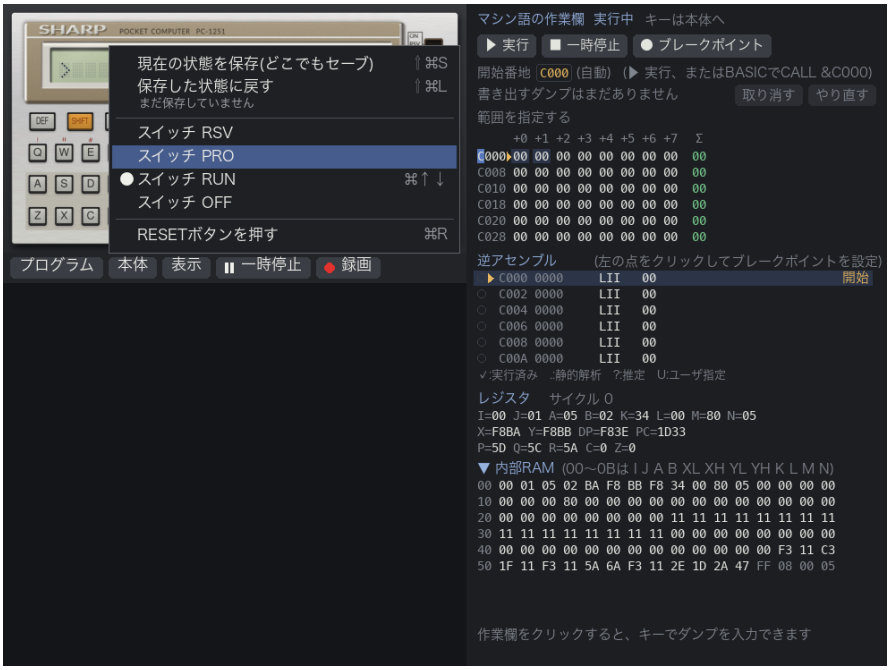
macOS	Windows	すること
command + E	Ctrl + E	BASIC のプログラムをファイルに書き出す (5.9 節)。作業欄にキーを送っているときは、作業欄のダンプを書き出す (7.7 節)
shift + command + E	Shift + Ctrl + E	作業欄にキーを送っているときに、作業欄のダンプをダンプリストの形で書き出す (7.7 節)
command + S	Ctrl + S	画面を画像で保存する
shift + command + S	Shift + Ctrl + S	状態を保存する (3.6 節)
shift + command + L	Shift + Ctrl + L	保存した状態に戻す
shift + command + R	Shift + Ctrl + R	録画を始める (前に選んだ形で)・止めて保存する (3.7 節)
shift + command + P	Shift + Ctrl + P	録画を一時停止する・再開する
command + P	Ctrl + P	実行を一時停止する・再開する
command + /	Ctrl + /	キーの説明を出す・消す
command + V	Ctrl + V	クリップボードの文字を打ち込む。作業欄にキーを送っているときは、ダンプを作業欄に貼り付ける (7.4 節)
command + K	Ctrl + K	矢印キーを数字キーの 8・2・4・6 にする・戻す
command + D	Ctrl + D	マシン語の作業欄を表示する・隠す (7.2 節)
command + Z	Ctrl + Z	作業欄の書き込みを取り消す (shift も押すとやり直す。7.4 節)
command + A	Ctrl + A	作業欄のカーソルの番地を開始番地にする (7.5 節)

同じことは、本体の画像の下の操作欄からもできます。操作欄の左のボタンを押すと、そのすぐ上にメニューが出ます。項目はクリックするか、↑ ↓ で選んで Enter で実行します。← → で隣のボタンのメニューへ移れます。

ボタン	メニューの中身
プログラム	プログラムの一覧、ファイルに書き出す、クリップボードから打ち込む、プログラムのフォルダを開く、作業欄の書き込みの取り消しとやり直し (作業欄を出しているとき)、読み込んだプログラムの説明
本体	いまの状態を保存、保存した状態に戻す、プログラムを読み込む前に戻す、スイッチ、RESET ボタン
表示・入力	キーの説明、矢印キーの割り当て、ゲームパッドの割り当て、マシン語の作業欄を表示する・隠す (`--no-basic` のときはない)、液晶だけを表示・本体の画像を表示、液晶の文字の間を詰める・実機と同じにする、画面を画像で保存、画面の言語 (English・日本語)、印字の保存先を開く、プリンタの紙を隠す

続く「実行を一時停止」は CPU の実行を一時停止するボタン、「速度」は実行速度を選ぶボタン (現在の速度が表示されます) です。右端の「● 録画」のボタンで、録画の形を選んで始める・一時停止する・止めることができます (3.7 節)。ボタンにマウスを載せると、何をするボタンかとショートカットが出ます。操作欄の右には、機種と、最後に読み込んだプログラムの名前が出ます。

ウィンドウの上で右クリックすると (Mac では control を押しながらクリックしても)、実行の一時停止、プログラムの一覧、画面を画像で保存、キーの説明だけの短いメニューが出ます。プリンタの紙の上で右クリックすると、紙の操作 (印字の保存先を開く、紙をしまう) が出ます。



command / Ctrl + P か操作欄の「実行を一時停止」を押すと、CPU もファイルの打ち込みも音も、その場で止まります。「実行を再開」か、もう一度 command / Ctrl + P を押すと続きから動きます。録画の一時停止 (shift + command / Ctrl + P) とは別の操作です。

テンキーのないパソコンでゲームをするとき

PC-1251 のゲームには、数字キーの 8 2 4 6 (5 を真ん中にした上下左右) で動かすものがあります。テンキーのないパソコンでは、command / Ctrl + K (または F7) を押すと、↑ ↓ ← → がそれぞれ 8 2 4 6 を押すようになります。もう一度押すと元に戻ります。プログラムを打ち込むときは、↑ ↓ が PC-1251 の ↑ ↓ として使えなくなるので戻しておいてください。起動のときに `--numpad` を付けると、矢印キーが数字キーになった状態で始まります。

液晶だけを大きく見せる

「表示・入力」のメニューの「液晶だけを表示」で、本体の画像を隠して液晶だけをウィンドウいっぱいに出せます。このあいだはウィンドウの大きさを変えられ、最大化すると液晶が画面いっぱいになります。キーはパソコンのキーボードかゲームパッドで押します。マシン語の作業欄はこのあいだ出せません。戻すには、操作欄の「本体を出す」を押します。起動のときに `--lcd` を付けると、液晶だけで始まります。

液晶の文字の間を詰める

「表示・入力」のメニューの「液晶の文字の間を詰める」で、桁と桁のあいだのすきまをなくし、液晶のドットを同じ間隔で並べて見せます。PC-1261 系では 2 行のあいだも詰めます。実機にはない見せ方で、液晶 RAM とプログラムの動きは変わりません。設定は次に起動したときも残ります。PC-1350 の液晶はもともとすきまがないので、この項目はありません。

速さを変えたときの音

1 倍以外の速さでは、音の高さも速さの倍率だけ変わります。2 倍なら BEEP は 1 オクターブ高く聞こえます。

ファンクションキー

F1〜F4(スイッチの RUN、PRO、RSV、OFF)、F5(RESET)、F6(一覧)、F7(矢印キーを数字キーに)、F9(速くする。Shift+F9で遅くする)、F10(説明)、F12(画面の保存) も使えます。Mac では fn を押しながら押します。

command/Ctrl+S で保存した画像は、ホームの `.sc61box` フォルダに `shot001.png`、`shot002.png`……の名前で置かれます (Windows では `C:\Users\名前\.sc61box`)。

3.5　ゲームパッドで操作する

OS につながっているゲームパッド (Bluetooth か USB) で、本体のキーを押せます。ペアリングは OS の設定で済ませておきます。エミュレータを起動したあとにつないでもかまいません。使えるのは、SDL の GameController の対応表にあるゲームパッドです (8BitDo、Xbox、PlayStation、Nintendo Switch のコントローラなど)。

十字キー、A B X Y、L R、ZL ZR、Select Start、スティックの押し込み、Home のそれぞれに、本体のキーを 1 つ割り当てます。左右のスティックは、倒した向きの十字キーと同じ入力になります。割り当てていない入力を押しても何も起きません。

最初の割り当ては、十字キーが 8 2 4 6、A が SPC、B が 0、Start が ENTER、Select が BRK です。

■割り当てを変える　「表示・入力」のメニューの「ゲームパッドの割り当て…」を開くと、本体の画像の下に割り当ての欄が出て、本体の画像のキーに、割り当ててある入力の札 (A、←など) が付きます。パッドのボタンを押すか欄の割り当てをクリックして入力を選び、本体の画像のキーをクリックすると、その入力の割り当てが変わります。英字・数字など SHIFT なしで打てるキーは、パソコンのキーを打っても割り当てられます。Delete か Backspace で割り当てを外します。Esc は、入力を選んでいるときは選ぶのをやめ、選んでいないときは欄をしまいます。欄を出しているあいだは、パッドの入力を本体に送りません。ボタンの名前 (A・B・X・Y) は SDL が付けたもので、パッドの印字と違うことがあります。そのときは、割り当てたいボタンを押して選びます。

■割り当てを保存する　割り当てはプログラムごとに持てます。欄のボタンで保存し、戻します。

- 「このプログラム用に保存」は、プログラムのフォルダの `pads/programs/プログラム名.json`(機種のフォルダにあるプログラムなら `pads/programs/pc1350/プログラム名.json` のように) に保存します。次にそのプログラムを読み込んだときは、ファイルの`# pad:` より優先します。
- 「既定として保存」は `pads/default.json` に保存し、`# pad:` のないプログラムと、起動したときに使います。
- 「初期設定に戻す」は最初の割り当てに、「# pad: に戻す」は読み込んだプログラムの`# pad:` の

割り当てに戻します。

- 「# pad: の行をコピー」は、いまの割り当てを# pad: の行としてクリップボードに写します。

保存した割り当てを消すには、そのファイルを消します。

■プログラムのファイルに割り当てを書く　プログラムのファイルに# pad: の行を書いておくと、読み込んだときにその割り当てになります。書き方は 5 節の指示の行の表にあります。

3.6　状態を保存する (どこでもセーブ)

shift+command/Ctrl+S を押すと、そのときの RAM、CPU のレジスタ、ポート、タイマ、スイッチの位置をまとめて保存します。shift+command/Ctrl+L を押すと、保存したところに戻って、そこから続けられます。操作欄の「本体」のメニューの「いまの状態を保存」「保存した状態に戻す」でも同じです。

保存先はホームの.sc61box フォルダの state-1251.json(PC-1245 なら state-1245.json のように機種ごと。ROM なしでは state-1251-norom.json のように別のファイル) です。保存し直すと上書きし、1 つ前の状態は state-1251.json.bak に残します。テープ (5.11 節) の再生の位置は保存しません。起動のときに--state ファイルをつけると、保存した状態から始めます。

3.7　画面と音を録画する

操作欄の右端の「● 録画」を押すと、録画の形を選ぶメニューが出ます。「液晶の拡大と本体」「本体だけ」「液晶だけ」のどれかを選ぶと、録画を始めます。shift+command/Ctrl+R を押すと、前に選んだ形で始めます (はじめは「液晶の拡大と本体」)。録画しているあいだは、そのボタンが「録画中」と経過時間の表示に変わります。このボタンを押すと、すぐ上に停止と一時停止のメニューが出ます。「録画を止めて保存」(または shift+command/Ctrl+R) で録画を止めて、動画のファイルに保存します。「録画を一時停止する」(または shift+command/Ctrl+P) のあいだは録りません。

動画は 1280×720 の MP4 のファイルで、左上に題名が入り、その下に選んだ形の絵を置きます。「液晶の拡大と本体」は上に液晶を拡大した絵、その下に本体全体の絵が並びます。「本体だけ」は本体全体の絵を、「液晶だけ」は液晶を拡大した絵を、中ほどに置きます。音も入ります (--mute で起動したときも入ります)。ファイルはプログラムのフォルダ (~/.sc61box/programs) に「rec-日時.mp4」の名前で置かれます。

動画を作る ffmpeg は、uv がライブラリと一緒にインストールします。Windows、macOS、Linux(x86_64 と aarch64) 以外では、ffmpeg を別にインストールして PATH に置きます。

3.8　モードスイッチ

モード	すること
RUN	計算とプログラムの実行。
PRO	プログラムを打ち込む・直す・LIST で見る。
RSV	よく使う文字列をキーに登録する (4.6 節)。
OFF	電源を切る。プログラムと変数は残る。

3.9　SHIFT と DEF

SHIFT は、押して離してから次のキーを押します (同時に押す必要はありません)。押すと表示記号 SHIFT が点き、次のキーがキーの上に赤く刷ってある字になります。たとえば SHIFT W で"、SHIFT 0 で π です。

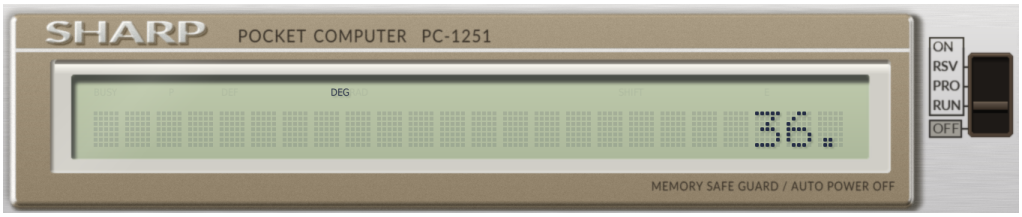
DEF は、プログラムの中の"A"のような印の行から実行を始めるキーです (4.5 節)。

4　使ってみる

4.1　電卓として使う

RUN モードで式 (たとえば 12*3) を打って ENTER を押すと、答えが出ます。

```
12*3
```



CL (パソコンでは Delete) で表示が消えます。

4.2　BASIC のプログラムを打ち込んで動かす

1. スイッチの PRO をクリックするか、command/Ctrl+↑で PRO モードにします。
2. 前のプログラムを消したければ NEW と打って ENTER を押します。
3. 行番号をつけて、1 行ずつ打ち込みます。行の終わりで ENTER を押します。

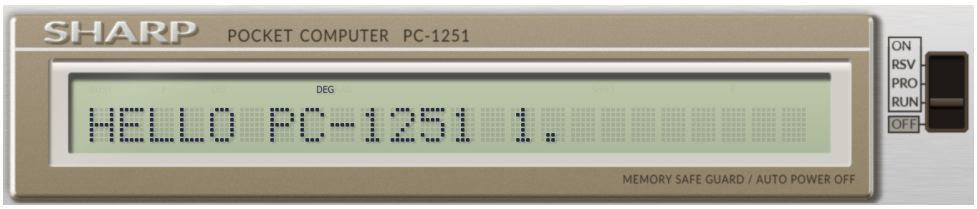
```
10 FOR I=1 TO 3
20 PRINT "HELLO PC-1251 ";I
30 NEXT I
```

```
40 END
```

4. ↓を押すと先頭の行が出て、押すたびに次の行へ進みます。↑で前の行に戻ります (何も出ていないときに押すと、最後の行が出ます)。`LIST 30` のように打つと、30 行から見られます。



5. command/Ctrl+↓で RUN モードに戻り、`RUN` と打って ENTER を押します。



PC-1251 の `PRINT` は、表示したところで止まって待ちます。ENTER を押すと先へ進みます (止めずに進める方法は次の節)。実行を途中で止めるには BRK (パソコンでは Esc) を押します。

長いプログラムを手で打つのは大変なので、ふつうはテキストファイルに書いておき、5 節の方法で読み込むのが楽です。

IF のあとの代入には LET がいる

`IF` の条件のあとに代入を書くときは、`LET` を省けません。`THEN` をつけても同じです。

書き方	結果
`IF S>H LET H=S`	代入される
`IF S>H THEN LET H=S`	代入される
`IF S>H THEN H=S`	`ERROR 4`
`IF S>H H=S`	エラーは出ないが、代入されない

最後の書き方は止まらずに進んでしまうので、間違いに気づきにくいところです。代入でなければ、`IF A=1 GOTO 50` や `IF D<>1 POKE &C5DF,0` のように、条件のすぐあとに命令を書けます。

4.3　打ち込みを楽にする、行を直す・消す

■命令は略して打てる　命令の頭の何文字かのあとに.(ピリオド) を打つと、打った文字で始まる命令になります。たとえば `P."AB"` と打てば `PRINT "AB"` と同じです。PRO モードでも RUN モードでも使え、`LIST` すると元の綴りで表示されます。

略記	命令	略記	命令	略記	命令
P.	PRINT	G.	GOTO	GOS.	GOSUB
I.	INPUT	F.	FOR	N.	NEXT
RE.	RETURN	REA.	READ	RES.	RESTORE
R.	RUN	L.	LIST	T.	THEN
B.	BEEP	W.	WAIT	CA.	CALL
D.	DIM	E.	END	S.	STOP

同じ文字で始まる命令がいくつかあるときは、ROM の命令の表で先に載っているものになります。C. は CALL ではなく CONT に、RE. は READ ではなく RETURN になるので、ほしい命令にならないときは文字を 1 つ増やします。.bas のファイルに略記で書いても同じように読み込め、5.9 節の方法で書き出すと元の綴りに戻ります。

■行を消す　PRO モードで行番号だけを打って ENTER を押すと、打った番号の行が消えます。たとえば 20 ENTER で 20 行が消えます。プログラム全体を消すときは NEW です。同じ行番号で打ち直すと、前の行と入れ替わります。

■行を直す　PRO モードで LIST 30 のように行を出し、▶ を押すと、行番号のすぐあとの文字にカーソルが出ます。

- ◀ ▶ でカーソルを動かし、文字を打つとカーソルの位置の文字を上書きします。
- SHIFT ▶ (INS) でカーソルの位置に 1 字ぶんの空きを作り、空いたところへ文字を打ち込みます。
- SHIFT ◀ (DEL) でカーソルの位置の文字を消します。
- 直し終わったら ENTER を押すと、直した行に入れ替わります。
- ↓ ↑ で次の行・前の行を出せます。

■パソコンのキーボードで打つとき　Backspace (macOS では delete) を押すと、エミュレータが ◀ と SHIFT ◀ (DEL) を続けて押し、カーソルの 1 つ前の文字を消します。打ち間違えたらそのまま Backspace で戻れます。

4.4　PRINT で止めずに見せる (WAIT)

PRINT のたびに ENTER を押すのでは、表示を動かすことができません。WAIT を使うと、PRINT は決まった時間だけ表示してから、止まらずに次へ進みます。

書き方	PRINT の動き
WAIT 64	表示してからおよそ 1 秒待ち、次へ進む。数は 10 進法で、大きいほど長く見せる
WAIT 0	待たずにすぐ次へ進む。表示は目に見えないほど短い
WAIT	数を書かないと、ENTER を待つもとの動きに戻る

一度 WAIT を実行すると、以降の PRINT はすべて同じ待ち時間で動きます。次の例は、POCKET COMPUTER の文字を右から左へ 1 字ずつ流し、左端まで来たところで止まります (見本の

anime.bas)。

```
10 DIM T$(0)*40
20 T$(0)="                          POCKET COMPUTER"
30 WAIT 2
40 FOR I=1 TO 24
50 PRINT MID$ (T$(0),I,24)
60 NEXT I
70 WAIT:PRINT "POCKET COMPUTER"
80 END
```



ふつうの文字列変数 (A$など) には 7 字までしか入らないので、10 行の DIM T$(0)*40 で、40 字まで入る文字列の配列 T$を作っています。かっこの中の 0 は添字の最大値で、使えるのは T$(0) の 1 つだけです。*40 は 1 つに入る字数で、書かなければ 16 字です。20 行では、T$(0) に空白を 24 字入れ、続けて流したい文字を入れておきます。50 行の MID$で I 字目から 24 字を切り出して表示し、I を 1 つずつ増やすと、文字が 1 字ずつ左へ動きます。30 行の WAIT 2 で、1 コマが 0.2 秒ほどになります。

PC-1251 の BASIC には、液晶の点を 1 つずつ描く命令がありません。動きのある表示は、文字を PRINT し直して作ります。点で絵を描きたいときは、液晶の RAM に直接書くマシン語を使います (5.6 節の見本)。液晶に 1 列ずつ点を打てる PC-インタープリタ (5.7 節) の DEG も使えます。PC-インタープリタの PRINT は止まらないので、WAIT は時間待ちの命令として使います。

4.5　DEF キーで実行する

行の頭に"A"のような印をつけておくと、RUN モードで DEF A と押すだけで、印をつけた行から実行できます。

```
10 "A" PRINT 5
20 END
```

DEF A で 5 が出ます。印に使えるのは、RSV モード (4.6 節) と同じ灰色の帯の上の 18 個のキー (A〜L、=、Z〜M、SPC) です。

4.6　RSV モードで文字列を登録する

よく打つ BASIC の命令や式を、キーに登録しておけます。

1. スイッチの RSV をクリックして RSV モードにします。
2. SHIFT A と押すと、液晶に A: と出ます。
3. 登録したい文字列 (たとえば PRINT や 12*3) を打って、ENTER を押します。



4. RUN モードか PRO モードで SHIFT A と押すと、登録した文字列が打ち込まれます。

登録に使えるのは、キーボードの下 2 段の灰色の帯の上にある、A〜L と =、Z〜M と SPC の 18 個です。帯の外のキー (Q など) には登録できません。

4.7　エラーが出たら

打ち込んだ式やプログラムにまちがいがあると、ERROR と番号が出て、表示記号 E が点きます。CL で消えます。



RUN モードで PRINT 3 と打つと ERROR 1 になる

RUN モードで PRINT を式の前につけて直接打つと、ERROR 1 になります。実機と同じ動きです。電卓として使うときは 12*3 のように式だけを打ちます。プログラムの中の PRINT は、ふつうに動きます。

4.8　電源とメモリ

電源を切る

スイッチの OFF をクリックします。

電源を入れる

スイッチを RUN か PRO に戻します。

オートパワーオフ

何もしないで 11 分ほどたつと、電源が切れます (実機と同じ)。BRK (ON) を押せば入ります。プログラムは残っています。

メモリの保存先

終わるときに、RAM と CPU の中のメモリを、ホームの `.sc61box/ram.bin` に保存します。ほかの機種では `ram-1245.bin` のように機種の番号を付けた名前、PC-1350 に RAM カードを差したときは `ram-1350-16k.bin` のように大きさも付けた名前です。

RESET ボタン

command / Ctrl + R で本体の裏の RESET ボタンを押せます。プログラムが暴走して BRK でも止まらないときに使います。押すと BASIC に戻り、プログラムと変数は残ります (実機では消えることがあります)。

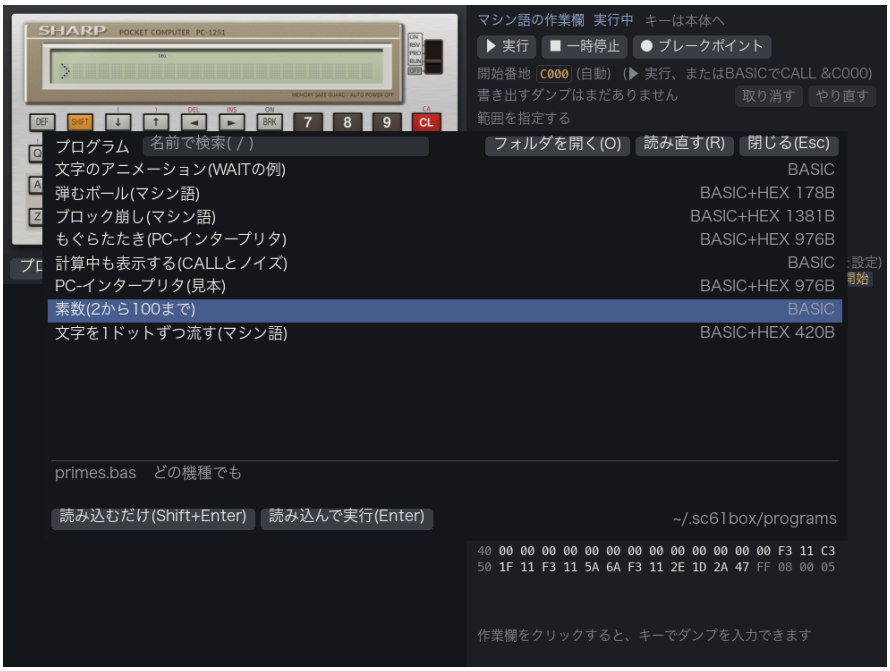
5　プログラムのファイルを使う

5.1　プログラムのフォルダに置いて、一覧から読み込む

BASIC のプログラムやマシン語のダンプを、決まったフォルダ (プログラムのフォルダ) にファイルとして置いておくと、command / Ctrl + O で開く一覧に出てきます。行をクリックして選び、下の「読み込んで実行」か「読み込むだけ」のボタン (または Enter、Shift + Enter) で読み込みます。選んだ行の下には、ファイル名と動く機種、ファイルに書いた説明 (`# help:`) が出ます。一覧の上の欄に名前の一部を打つと、合うものだけに絞れます。

macOS	`/Users/名前/.sc61box/programs`
Windows	`C:\Users\名前\.sc61box\programs`

プログラムのフォルダは、最初に一覧を開いたときにできます。一覧で O を押すと、Finder やエクスプローラーで開きます。



キー	すること
↑ ↓	選ぶ (マウスでクリックしても選べる)
ホイール、PageUp PageDown	一覧を動かす (入りきらないときは、右にバーと「▼」が出る)
Enter	読み込んで実行する (探しているときも同じ)
Shift + Enter	読み込むだけ (打ち込んだプログラムを直したいときなど)
O	プログラムのフォルダを開く
R	一覧を読み直す (ファイルを足したあと)
/	名前で検索 (続けて名前の一部を打つ。Esc で検索を終了)
Esc	一覧を閉じる

プログラムのフォルダのほかに、エミュレータのフォルダの programs にある見本 (素数、文字のアニメーション、計算中の表示、文字を 1 ドットずつ流す、弾むボール、ブロック崩し、PC-インタープリタ、もぐらたたき、PC-1261 の 2 行の表示、PC-1350 の動く線とヘビ) も一覧に並びます。同じ名前のファイルが両方にあるときは、プログラムのフォルダのほうが使われます。

■機種ごとのフォルダ　プログラムは、動く機種に合わせて次のように置きます。

- 1 つの機種だけで動くものは、プログラムのフォルダの中の機種のフォルダ (pc1251、pc1245、pc1261、pc1350) に置きます。pc1251 に置いたものは PC-1250 と PC-1255 でも、pc1261 に置いたものは PC-1260 と PC-1262 でも、マシン語の番地が RAM に収まり、BASIC のプログラムの領域にかからなければ使えます。
- いくつかの機種で動くものは、プログラムのフォルダの直下に置き、ファイルに# model: 1251,1245 のように動く機種を書きます。
- どの機種でも動く BASIC は、直下に# model: を書かずに置きます。

直下に置いたもので# model: のないものは、マシン語と POKE の番地が RAM か液晶に収まる機種で使えるとみます。

```
programs/
    primes.bas          どの機種でも
    pcint.bas           # model: 1251,1245
    mogura.bas          # model: 1251,1245
    pc1251/
        break.bas       PC-1251 用
        break.hex
    pc1245/
        break.bas       PC-1245 用
        break.hex
    pc1261/
        hello.bas       PC-1261 用 (PC-1260、PC-1262 でも)
    pc1350/
        hello.bas       PC-1350 用
```

一覧には、いま動かしている機種で使えるものだけが並びます。機種ごとに作り分けたプログラム

は、同じ `title` でそれぞれのフォルダに置いておけば、どの機種でも一覧の同じ名前から読み込めます。

■ドラッグ&ドロップ　Finder やエクスプローラーからファイルをエミュレータのウィンドウにドロップしても、同じように読み込めます。

5.2　ファイルの形式

ファイル	読み込み方
名前.bas	BASIC のテキスト。PRO モードにして NEW してから、RAM に書き込む。長いプログラムもすぐに入る。
名前.hex	16 進ダンプ。番地 データの行を読んで、RAM にそのまま書き込む。
名前.bas と名前.hex	同じ名前の 2 つは 1 本のプログラムとして扱い、ダンプを書いてから BASIC を打ち込む。マシン語と、マシン語を呼ぶ BASIC を組にしておける。

■BASIC のファイル　ふつうのテキストファイルです。文字コードは UTF-8(ASCII の範囲なら何でも同じ) で、改行は macOS・Windows どちらの形でもかまいません。小文字で書いても大文字に直して読み込みます。PC-1251 のキーにない字は捨てます。円周率の π (SHIFT 0 で入る字) は、ファイルの中でも π と書きます。空の行と、#で始まる行は読み込みません。

■指数の書き方　PC-1251・PC-1250・PC-1255・PC-1245 では、指数を表す E(SHIFT + で入る Exp の字) は英字の E とは別の字なので、ファイルの中では [E] か€ と書きます。たとえば 1×10^{-8} は 1[E]-8 と書きます。1E-8 や 1e-8 と書くと英字の E になり、「1×E − 8」(変数 E との掛け算) になります。PC-1261(PC-1260・PC-1262 も) と PC-1350 には Exp のキーがないので、指数は英字の E で 1E-8 と書き、[E] と€ も英字の E として打ちます (10 節、11 節)。

■行の長さ　実機で 1 行に打てるのは 79 字までですが、行を呼び出して打ち足せば実機に入る長さの行なら、79 字を超えていても読み込みます。

```
# title: 素数 (2 から 100 まで)
# PRINT で止まるので、ENTER で次を出す
10 PRINT 2
20 FOR N=3 TO 100 STEP 2
30 D=3
40 IF D*D>N GOTO 70
50 IF N=INT (N/D)*D GOTO 80
60 D=D+2:GOTO 40
70 PRINT N
80 NEXT N
90 END
```

■16 進ダンプのファイル　1 行に番地とデータを 16 進法で書きます。番地は 4 桁です。データは空白で区切っても続けて書いてもかまいません。雑誌のダンプリストのように行の終わりに: とチェックサム (行のバイトの和の下 2 桁) がついていれば、バイトの和と比べます。合わない行や、: のあとが 16 進表記の 2 桁でない行があれば、その行の番号を出して読み込みません。#か; で始まる行はコメントです。行の途中でも; から後ろはコメントなので、ニーモニックを書き添えておけます。

```
# title: マシン語の見本
C300 02 41 37
C303 0102030405060708    ; 表の中身
```

アセンブラ YASM61860 が出力する 16 進ダンプ (c300 : 02 41 37 …の形の行) も、そのまま.hex として読めます。

■マシン語のバイナリ (.bin)　.bin のファイルは、.bas のファイルに# bin: ファイル名 番地の行を書き、置く番地を指定して読み込みます。番地は&C300、0xC300、C300 のどの形式で書いてもかまいません (どれも 16 進表記です)。

```
# title: マシン語の見本 (バイナリ)
# bin: sample.bin &C300
10 CALL &C300
```

YASM61860 の-r の.bin は番地 0 から始まり、ORG より前の RAM まで 0 で書きつぶすので、16 進ダンプ (-d) を.hex として使ってください。

■指示の行　#で始まる行には、次の指示を書けます。.bas と.hex のどちらに書いてもかまいません。

書き方	意味
# title: 名前	一覧に出す名前。書かなければファイル名。
# run: 文字列	一覧から選んだときや--run で実行するときに、RUN モードで打つ文字列。書かなければ RUN。いくつも書くと上から順に打ち、2 つ目からは、前の命令が終わって入力待ちに戻るのを待ってから打つ。
# mode: PRO	# run: の文字列を PRO モードで打つ。PRO モードで動かすマシン語のため。
# help: 説明	読み込んで動かし始めたときに、画面の下に出す説明 (遊び方など)。いくつも書くと行を重ねて出す。
# setup: 命令	NEW のあと、BASIC を打ち込む前に PRO モードで打つ命令。これがあるときは、ダンプも NEW とこの命令のあとで書き込む。EQU#で BASIC のプログラムの領域を動かしてから、空いたところにマシン語を置く記事のためのもの。
# hex: ファイル名	先に書き込む 16 進ダンプ。同じフォルダ、プログラムのフォルダの同じ機種のフォルダ、プログラムのフォルダの直下の順に探す。1 つのマシン語を、いくつもの BASIC のプログラムから使える。

書き方	意味
`# bin: ファイル名 番地`	先に書き込むマシン語のバイナリ (`.bin`) のファイル名と、置く番地。ファイルを探す場所と順番は`# hex:` と同じ。
`# after: ファイル名`	実行の文字列を打ったあとで書き込む 16 進ダンプ。BASIC で `DIM` などをして場所を空けてからマシン語を読み込む記事のためのもの。`.bin` を書き込むときは、`# after: ファイル名.bin 番地`のように番地も書く。
`# data: 範囲`	命令ではなくデータの範囲 (7.6 節)。`C100-C17F` のように始めと終わり (終わりの番地も含む) を書き、いくつもあれば`,`で区切る。終わりを書かずに `C100` とすると、そこから次の指定か実行した命令の手前までをデータにする。マシン語の作業欄の逆アセンブルで `DB` の行にする。
`# model: 機種`	プログラムが動く機種 (`1251`、`1245`、`1261` など。いくつもあるときは`,`で区切る。`1261` と書いたものは PC-1260 と PC-1262 でも動く)。書かなければ置いたフォルダで決まる。
`# pad: 割り当て`	ゲームパッドの割り当て (3.5 節)。`dpup=8 dpdown=2 dpleft=4 dpright=6 a=SPC` のように、パッドの入力の名前と本体のキーの名前を`=`でつなぎ、空白で区切って並べる。入力の名前は `dpup`・`dpdown`・`dpleft`・`dpright`・`a`・`b`・`x`・`y`・`leftshoulder`・`rightshoulder`・`lefttrigger`・`righttrigger`・`back`・`start`・`leftstick`・`rightstick`・`guide`。キーの名前は、いまの機種のキーの名前で、`SPC`・`ENTER`・`UP`・`DOWN`・`LEFT`・`RIGHT`・`SHIFT`・`DEF`・`BRK` と英数字・記号 (PC-1350 以外では `CL`、PC-1350 では `CLS`・`MODE`・`KANA`・`INS`・`DEL` も)。`,` は区切りではなく、PC-1350 の`,`のキーの名前として書ける (`a=,`)。書いた入力だけが割り当てられ、ほかの入力は何もしない。いくつかの行に分けて書いてもよい。
`# norom: 説明`	ROM なしで読み込んだとき (6 節) に、画面の下に出す説明。`# norom: off` と書くと、ROM なしの一覧に出さない (ROM のルーチンを使うものなど)。`# norom: basic` と書くと、ROM なしの BASIC では一覧に出し、マシン語の作業欄だけのとき (`--no-basic`) は出さない。
`# ram: 範囲`	機種の RAM の終わりより上も RAM にする。`6800-7FFF` のように書くと、終わりの番地まで RAM を広げる (実機にない番地に RAM を置いて動かすプログラムのため。ROM や液晶 RAM にかかる範囲は書けない)。読み込むと広がり、終わるときの RAM と一緒に保存する。`# ram:` のないプログラムを読み込むと、機種の RAM に戻る。
`# card: 大きさ`	PC-1350 で要る RAM カードの大きさ (KB。`8` か `16`)。差したカードが小さいときは読み込まず、`--card` をつけて起動するように知らせる。

`# hex:` や`# after:` で使われるだけの`.hex`(同じ名前の`.bas` がないもの) は、一覧には出しません。`.bin` のファイルは、どれも一覧には出ません。ダンプの大きさは、使う側のプログラムの行に出ます。

読めない行やファイル、RAM に入らないプログラムがあるときは、何も書き込まず、画面の下にその行やファイルを出します。

NEW で前のプログラムは消える

.bas を読み込む前に NEW するので、入っていたプログラムは消えます。残しておきたいプログラムは、先に 5.9 節の方法でファイルに書き出しておいてください。
読み込んだあとでも、操作欄の「本体」のメニューの「プログラムを読み込む前に戻す」で、最後に読み込む直前の状態に戻せます。

5.3　端末から読み込む

起動するときにプログラムの名前かファイルの場所を書くと、起動してすぐに読み込みます。名前だけならプログラムのフォルダから探します。--run をつけると、読み込んだあと実行します。

```
$ uv run sc61box primes --run
$ uv run sc61box ~/basic/game.bas
```

Windows なら、ファイルの場所は C:\Users\名前\basic\game.bas のように書きます。

5.4　マシン語を動かす

マシン語は 16 進ダンプのファイルにして RAM に書き込み、BASIC の CALL で呼びます。PC-1251 の RAM は B800〜C7FF 番地の 4KB で、BASIC のプログラムは下の番地から、変数は上の番地から使います。マシン語は、BASIC のプログラムや変数とぶつからない番地に置いてください。マシン語を打ち込んで 1 命令ずつ調べるときは、マシン語の作業欄 (7 節) を使います。

5.5　CALL で表示をつけたまま計算する

この節は ROM 使用時の例です (見本の noise.bas は、ROM なしの一覧には出ません)。BASIC はプログラムを実行しているあいだ、液晶の表示を切っています (C ポートのビット 0 を 0 にする)。マシン語でビット 0 を 1 にしてから計算を続けると、実行中も表示が消えません。見本の noise.bas は、表示をつけるマシン語 5 バイトを POKE で書いて CALL で呼び、計算しながら液晶に棒を描きます。

```
10 POKE &C200,2,1,255,177,55
20 CALL &C200
30 FOR I=0 TO 47
40 B$=STR$ (I*I)
50 POKE &F87B-I,127
60 NEXT I
70 WAIT 128:PRINT "DONE"
80 END
```

10 行で書くマシン語は次の 3 命令です。1FB1 番地は CPU の内部 ROM にあるルーチンで、A の値を C ポートに出します。

```
C200  02 01    LIA 01      A=&01(表示をオン)
C202  FF B1    CAL 1FB1    C ポートに A を出す
C204  37       RTN         BASIC に戻る
```

50 行は、液晶 RAM に直接 1 列ずつ点を書きます。液晶の右半分は F87B 番地が左端 (60 列目) で、番地が小さくなるほど右へ進むので、&F87B-I で棒が右へ伸びます。



表示をつけたままにすると、左の数桁に不規則な点の模様が出ます。ROM が液晶 RAM の一部を計算の作業場所に使っているためです。

CALL するマシン語を置く番地

C200 番地は PC-インタープリタ (5.7 節) が使う領域の中にあり、C000 番地から置くマシン語の見本とも重なります。続けて使うときは、あとから読み込んだ方が残ります。

5.6　マシン語で液晶に描く

■文字を 1 ドットずつ流す　見本の scroll.hex と scroll.bas は、POCKET COMPUTER と機種名を、液晶の全幅で右から左へ 1 列ずつ流すマシン語です。4.4 節のように PRINT で流すと 1 文字 (5 列) ずつ跳びますが、液晶 RAM に直接書けば 1 列ずつ動かせます。.hex には 1 行ごとに ; のあとへニーモニックを書いてあります。Esc (BRK) を押すと BASIC に戻ります。PC-1251 用は C000 番地から、PC-1245 用は C100 番地から置きます。液晶の幅が違うので、2 つは別々のマシン語です。

■弾むボール　見本の ball.hex と ball.bas は、ボールが地面でひしゃげ、放物線を描いて弾みながら、液晶の全幅を右へ進み、右端まで行くと左端から出てくるマシン語です。1 回の弾みを 9 枚の絵にして表に持ち、1 枚ずつ液晶 RAM へ写します。PC-1251 用は C000 番地から、PC-1245 用は C100 番地から置き、違うのは液晶の幅だけです。PC-1261 用は、2 行の液晶を使う別のマシン語です (10 節)。

■ブロック崩し　見本の pc1251/break.hex と break.bas は、横向きのブロック崩しです。左端のパドルを ↑ ↓ で動かし、Space か Enter で玉を打ち出して、右のブロックの壁を削ります。壁と画面の右端のあいだは空いているので、空いたところへ玉を回り込ませると壁を裏から削れます。壁はゆっくり左へ寄ってきて、先端が左から 12 列目まで来ると押し切られ、玉を 1 個失います。玉は 3 個で、全部なくなると BASIC に戻って得点を出します。右端の 3 つの点が残りの玉、残りの玉の右の 5 桁が得点です。

始めに 1:EASY 2:NORMAL ?と聞いてくるので、1 か 2 を押して ENTER を押します。EASY はパ

ドルが縦 3 段で、玉は 1 コマに 1.5 列、壁は 60 コマで 1 列と遅めです。NORMAL はパドルが縦 2 段で、玉は 1 コマに 2 列、壁は 40 コマで 1 列です。ENTER だけを押すと前と同じ難易度で始まります。最高点は難易度ごとに覚えています。ゲームパッド (3.5 節) では、十字キーの↑↓でパドルを動かし、A で玉を打ち出し、始めの難易度は X で 1、Y で 2 を選んで Start で ENTER を押します (`break.bas` の`# pad:` の行の割り当て)。



パドルに当てた場所で玉の角度が変わり、パドルを動かしながら当てるとさらに角度がつきます。縦が 7 ドットしかないので、同じ角度のまま打ち返していると、同じ段ばかりを往復して最後の数個が残りがちです。角度を変えて狙ってください。面をクリアすると 500 点が入り、次の面から玉が少し速くなります。

面ごとに 1 つ、点滅しているブロックがあります。点滅しているブロックを壊すと、壊した場所からもう 1 個の玉が出て、2 個で遊べます。玉は 2 個までです。新しい玉は壊した玉と同じ向きに進み、壊した玉のほうが跳ね返るので、2 個はブロックの場所から別々の向きへ進みます。片方を落としても残りの玉は減らず、2 個とも落とすと 1 個失います。

マシン語は `C000`〜`C56E` 番地で、作業用に `BF84` 番地から上を使うので、BASIC のプログラムは `BF83` 番地までに収めます。終わったあと、得点の 5 桁は `C5F6`〜`C5FA` 番地から `PEEK` で読めます。難易度は、`CALL` の前に `C5DF` 番地へ `POKE` しておきます (1 なら EASY)。PC-1245 用 (9 節) も、得点の退避先と難易度の番地は同じです。

5.7　PC-インタープリタともぐらたたき

見本には、PiO 1986 年 8 月号の「PC-インタープリタ」(Fan_PC-1251 氏作) が入っています。BASIC のエディタで書いた行を独自の文法で読んで実行するマシン語のプログラムで、数はすべて 16 進法 2 桁の 1 バイト、命令は `PRINT`、`IF`〜`NOT`、`GOTO`、`FOR`〜`NEXT`、`BEEP`、`WAIT`、液晶に 1 列ずつ点を打つ `DEG` などです。

`pcint.hex` と `pcint.bas`

記事のリスト 1(`C200`〜`C5CF` の 976 バイト) と、PC-インタープリタを試す短いプログラムの組です。一覧から選ぶと、`CALL &C300` で動き出します。

`mogura.bas`

記事のリスト 2 のもぐらたたきです。`# hex: pcint.hex` の行で PC-インタープリタを先に書き込み、`RUN 900` で始めます。

`pcint.hex` と `mogura.bas` は、Fan_PC-1251 氏の作品で、エミュレータの MIT ライセンスには含まれず、作者の了解を得て同梱しています。



もぐらたたきを始めるには、command / Ctrl + O (または操作欄の「プログラム」のメニューの「プログラムの一覧」) で開く一覧から「もぐらたたき (PC-インタープリタ)」を選びます。PC-インタープリタのダンプとリスト 2 が読み込まれ、RUN モードで `RUN 900` が打たれてゲームが始まります。一度読み込んだあとは、RUN モードで `RUN 900` と打つか、DEF = を押せば何度でも遊べます (900 行の頭に`"="`の印がある)。

900 行は `CALL &C300` で PC-インタープリタを呼び、ゲームが終わって BASIC に戻ってから `PRINT` で得点を出します。`CALL &C300` とだけ打っても遊べますが、得点は出ません。

PC-インタープリタともぐらたたきは、ROM なしの BASIC(6 節) でも動きます。PC-インタープリタが呼ぶ ROM のルーチンは、エミュレータが代わりに受け持ちます。

遊び方は次のとおりです。液晶に 1〜6 の穴が並び、どこかの穴からもぐらが 3 段階で顔を出します。出た穴の数字キーを押しっぱなしにしておくと、当たりの音が鳴って得点が 1 つ増え、次のもぐらが出ます。当てるたびに少しずつ速くなります。3 段目まで出きっても押せないと `GAME OVER` になり、ENTER を押すと得点が出ます。

記事のプログラムの扱い

`pcint.hex` と `mogura.bas` は記事の作者 (Fan_PC-1251 氏) の著作物で、作者の了解を得て収めています。PC-インタープリタを 1 命令ずつ解説した『PC-インタープリタを読む』(おりぐち) も `doc/pcinterp.pdf` にあります。

5.8　ほかの読み込み方

command + V (Windows は Ctrl + V)

クリップボードの文字を、そのままキーとして打ち込みます。`NEW` もモードの切り替えもしないので、打ち込む前に PRO モードにしておきます。

`--type ファイル`

起動したあと、ファイルの中身をそのままキーとして打ち込みます。

`--load ファイル`

起動したあと、16 進ダンプを RAM に書き込みます。

5.9　プログラムをファイルに書き出す

エミュレータで打ち込んだり直したりした BASIC のプログラムは、ファイルに書き出して残せます。command / Ctrl + E を押すか、操作欄の「プログラム」のメニューで「プログラムをファイルに書き出す」を選ぶと、いまのプログラムをプログラムのフォルダ (`~/.sc61box/programs`) に `.bas`

のファイルとして書き出します。ファイル名は「名前-日時.bas」で、名前は最後に読み込んだプログラムのもの (なければ `program`) です。最後に読み込んだものが機種ごとのフォルダにあったときは、プログラムのフォルダの同じ名前のフォルダ (`pc1245` など) に書き出します。書き出したファイルはすぐに一覧に出るので、そのまま読み込み直せます。

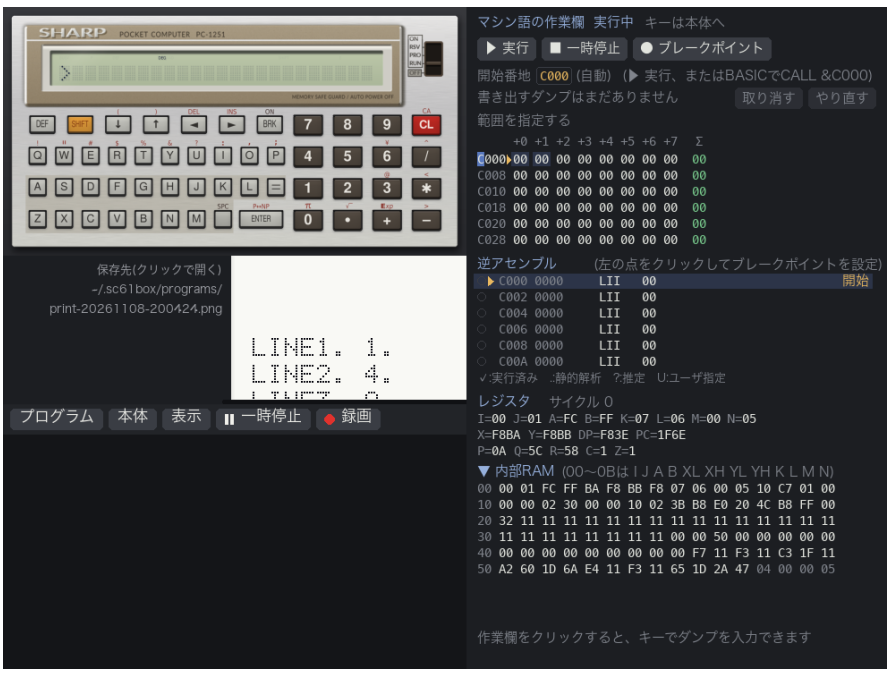
読み込んだプログラムに組になる `.hex` があったときは、書き出したファイルの頭に `# hex:` の行を入れて、元のダンプを指しておきます。`# bin:` や `# after:` で `.bin` を読み込んでいたときは、同じ `# bin:`・`# after:` の行 (番地つき) を頭に入れておきます。`POKE` で書いたマシン語など RAM の中身そのものは書き出しません。

エミュレータを起動しないで、保存してある RAM から書き出すこともできます。

```
$ uv run sc61box --export ~/Desktop
```

5.10　プリンタに印字する

エミュレータには、プリンタ (CE-126P にあたるもの) がいつもつながっています。`LPRINT` や `LLIST` で印字すると、本体の画像の下に紙が出てきて、行が 1 つずつ印字されて紙が送られます。印字の速さは実機と同じ 1 行 1 秒ほどで、速さの設定 (command / Ctrl + T) を変えると同じ割合で変わります。長いプログラムを `LLIST` で印字したときのように、まだ紙に出ていない行が 4 行以上あると、速めて 2 秒半ほどで出し切ります。紙送りの音は、CE-126P の音に似せて合成しています。音は `--mute` で消えます。



- 紙には最後の 5 行が見えます。紙の上でホイールを回すと、前の行を見られます。

- 印字は紙の絵として、プログラムのフォルダに `print-日時.png` の名前で自動で保存します。保存先は紙の左に出ます。保存先をクリックするか、メニューの「印字の保存先を開く」を選ぶと、そのフォルダを開きます。
- 紙の右の × か、操作欄の「表示・入力」のメニューの「プリンタの紙を隠す」で紙を隠します。しまうとその紙は終わりで、次に印字すると新しい紙 (新しいファイル) になります。
- SHIFT ENTER (P ↔ NP) を押しておくと、手で打った計算の式と答えも印字されます。PC-1251・PC-1245 では液晶に `P`、PC-1261 では `PRINT` が点きます。PC-1350 の液晶には印字モードの記号がありません (実機も同じ)。
- ROM なしで動かすときは、BASIC から打つ字を、エミュレータが液晶に描くのと同じ形で打ちます (6.1 節)。

プリンタは本体とは別に字の表を持っていて、コード `00`〜`FF` のすべてに字があります。小文字やカナのほか、≠ Σ θ → などの記号、小さい数字、年月日円などです。BASIC から打てるのはその一部で、ほかの字はマシン語でプリンタにコードを送ると打てます。

5.11　カセットの音 (wav) で実機とやり取りする

実機のカセットインターフェースで読み書きする音を、wav のファイルにして実機とやり取りできます。

■エミュレータから実機へ　RUN モードで `CSAVE` か `CSAVE M` を打つと、ROM がカセットに出す音を、エミュレータがそのまま wav のファイルに書き出します。ファイルはプログラムのフォルダ (`~/.sc61box/programs`) に「tape-日時.wav」の名前で置かれます。実機に読み込ませるときは、wav をパソコンで再生し、実機のカセットインターフェースの入力につないで、実機で `CLOAD`(マシン語は `CLOAD M`) を打ちます。書き出した wav を実機で読み込ませることは、まだ確かめていません。

```
CSAVE "BREAK"
CSAVE M "BREAK";&C000,&C56E
```

音が止まって 1 秒たつと wav を書き出します。前置きの音が 8 秒あるので、短いプログラムでも 10 秒ほどかかります (command / Ctrl + T で速くすると短くなります)。

■実機からエミュレータへ　実機で `CSAVE` した音をパソコンで録音して wav にし、ウィンドウにドロップすると、テープをセットした状態になります (起動のときに `--tape` ファイルを付けても同じ)。テープをセットしたあと、RUN モードで `CLOAD` か `CLOAD M` を打つと、ROM がテープを読み始めたところで再生が始まり、実機と同じ時間をかけて読み込みます (command / Ctrl + T で速くもできます)。

実機のテープを録音した wav が `CLOAD` で読めることは確かめてあります。PocketTools の `bin2wav` や、YASM61860 の `-w`(`-old` をつけたとき) で作ったマシン語の wav も、PC-1251 と PC-1245 のどちらでも `CLOAD M` で読めます。

6　ROM なしで使う

使う機種の ROM のファイルが見つからない場合 (大きさが足りない場合も) は、ROM なしで起動します。ROM を置いている場合でも ROM なしで使うには、`--norom` を付けて起動します。

```
$ uv run sc61box --norom
$ uv run sc61box --model 1350 --norom
```

ROM なしでは、どの機種もエミュレータに内蔵した BASIC(ROM なしの BASIC) で動作します。行の入力と編集、RUN モードの計算、プログラムの実行 (PRINT、INPUT、FOR、GOSUB、IF、DATA、USING、BEEP、TRON など)、RSV モード (PC-1245 にはありません) と DEF キー (4.6 節、4.5 節)、プリンタ (5.10 節)、パスワード (PASS)、自動電源オフ、カセットの音 (5.11 節) は、ROM 使用時と同じように使えます。PC-1350 では、グラフィックの命令、シリアル入出力の命令、TEXT・BASIC のモード (11.4 節、11.5 節、11.6 節) も使えます。作業欄は command / Ctrl + D で表示と非表示を切り替えます (7.2 節)。

command / Ctrl + O の一覧には、ROM 使用時と同じように BASIC のプログラムが表示され、読み込んで実行できます。ROM のルーチンを使うものなど、ファイルに `# norom: off` を指定したプログラムは表示されません。

6.1　ROM 使用時と違うところ

ROM 使用時とおもに違うのは、以下の点です。

- フォントは独自に作成したもので、ROM のフォントとはやや異なります。どの機種でも、フォントは ROM と同じ番地に同じ並びで置いているので、ROM のフォントを読むマシン語もそのまま使えます。印字する字のうち液晶にもある字は、ROM の有無にかかわらず、液晶と同じフォントで印字します。
- マシン語のプログラムから ROM のルーチンを呼んでいる場合は、エミュレータに代替 ROM ルーチンがある場合のみ動作します。代替 ROM ルーチンは、ROM のルーチンと同じ処理をして呼び出し元に戻ります。代替 ROM ルーチンが存在しない場合には、警告を出して止まり、BASIC に戻ります (プログラムの実行中なら `BREAK` で止まります)。警告は、呼んだ番地とともにウィンドウの左上に表示し、次にキーを押すと消えます。代替 ROM ルーチンがあるのは、PC-1251(PC-1250、PC-1255 も) と PC-1245 では、数値計算、文字列、キー入力、表示、時間待ちなど、よく呼ばれるルーチンです。PC-1261 系と PC-1350 では、レジスタの受け渡し、フォント、キー入力などのルーチンです。呼び出せる番地と、引数・戻り値の一覧は `doc/norom.md` にあります。
- 代替 ROM ルーチンのないルーチンは、利用者が Python で代替 ROM ルーチンを記述できます (6.2 節)。
- ROM の番地からデータを読み出すプログラムは、ROM なしでは正しく動作しません。ROM

の番地には、フォント (PC-1350 ではキーの表も) を除いて ROM の内容が入っていないためです。ROM の中の命令を探すプログラムも、同じ理由で動作しません。

- 実行速度は実機に近づけていますが、計算精度や計算時間は実機と異なることがあります。
- PC-1350 で TEXT を実行すると RAM に収まらなくなる場合、ROM 使用時はプログラムが壊れますが、ROM なしでは ERROR 6 になり、プログラムは変わりません。
- RAM のうち、プログラム、変数、よく読まれる番地以外の、ROM が作業に使う領域の内容は、ROM 使用時と同じとは限りません。
- 終了時の RAM は、ROM 使用時とは別のファイル (ram-1251-norom.bin など) に保存します。ファイルの形式は同じなので、名前を ROM 使用時の名前 (PC-1251 なら ram.bin、ほかの機種は ram-1245.bin など。4.8 節) に変えれば、ROM を使って起動したときにも読み込めます。

6.2　代替 ROM ルーチンを記述する

未対応の ROM ルーチンは、Python で実装して追加できます。代替 ROM ルーチンは、ホームの.sc61box フォルダの中の routines フォルダに、.py のファイルとして置きます。起動したときに読み、作業欄と ROM なしの BASIC の CALL で使います。同じ番地に組み込みの代替 ROM ルーチンがあるときは、mode="fallback"なら組み込みのほうを、mode="override"なら記述したほうを使います。--new-routine で作る書き始めのファイルは mode="fallback"です。

書き始めのファイルは、--new-routine で作れます。番地は 16 進で書き、: のあとに名前をつけられます。作業欄や ROM なしの BASIC が代替 ROM ルーチンのない番地で止まったときの知らせにも、このオプションが出ます。

```
$ uv run sc61box --model 1261 --new-routine 1234:TWICE
```

できたファイル (この例では routines/twice.py) の関数に、働きを書きます。関数は本体を受け取り、内部 RAM やメモリ、フラグを読み書きして、使った CPU のサイクル数を返します。働きは、ROM 使用時のエミュレータで値を与えて呼び、戻ったときの値を見て決めてください。ROM の命令の並びや表を写したものは置かないでください。書き方の詳しい説明は doc/norom.md の「利用者が記述する代替 ROM ルーチン」にあります。

記述した代替 ROM ルーチンは、--check-routines で、ROM 使用時の動きと比べられます (ROM のファイルが要ります)。同じ値を与えて 30 回呼び、内部 RAM、フラグ、メモリ、かかったサイクル数が違った回を知らせます。

```
$ uv run sc61box --model 1261 --check-routines user
```

user の代わりに名前 (コンマで区切って複数) を書くと、その代わりだけを、all と書くと組み込みの代替 ROM ルーチンも比べます。

sc61box の新しい版に同じ番地の組み込みの代替 ROM ルーチンが入ったあとも、記述した代

替 ROM ルーチンを使い続けるときは、`mode="override"`に書き換えます。番地や名前がほかの代替 ROM ルーチンとぶつかるものは使いません。どの代替 ROM ルーチンを使っているかは、`--list-routines` で一覧にできます。

6.3　作業欄だけで動かす

`--no-basic` をつけると、BASIC を付けずに、マシン語の作業欄だけで動きます。CPU、液晶、キー、ブザー、タイマは、ROM があるときと同じように使えます。液晶に>は出ず、マシン語は作業欄から実行します。マシン語が ROM の番地を呼ぶと、そこで止まります (7.5 節。代替 ROM ルーチンを持つルーチンは、6.1 節のとおり働きをまねます)。

```
$ uv run sc61box --norom --no-basic
```

一覧には、マシン語のダンプがあるプログラムだけが並びます。読み込むと、ダンプと、組になる BASIC の `POKE` を RAM に書き、BASIC の行は入れません。開始番地は、BASIC の最後の `CALL` の番地になります。読み込んだら command / Ctrl + G で実行します。見本の弾むボールとブロック崩しは、作業欄だけでも遊べます。開始番地やブレークポイントは、RAM と一緒に保存します。

7　マシン語の作業欄

マシン語の作業欄は、本体の画像の右に出る欄です。マシン語を 16 進ダンプで打ち込み、BASIC の `CALL` で呼んで実行し、1 命令ずつ進めて調べられます。ROM 使用時も ROM なしの BASIC でも、command / Ctrl + D で出し入れして使います (7.2 節)。ROM なしで、BASIC を付けずに作業欄だけで動かすこともできます (6.3 節)。



7.1　作業欄の各部

見出しとボタン

いちばん上に「マシン語の作業欄」(作業欄だけで動かすときは「マシン語 (ROM なし)」) と、いまの状態 (編集中、実行中、一時停止中) が出ます。その下のボタンは、状態に合わせて入れ替わります。

開始番地

実行を始める番地です。クリックすると、16 進表記の 4 桁を打って変えられます (7.5 節)。右の「実行の前に液晶を消す」に印がついていると、実行の前に液晶を消して、表示をつけます。

書き出し

「.hex に書き出す」「ダンプリスト」のボタンと、書き出す範囲のバイト数と番地が出ます (7.7 節)。同じ行の右端に、書き込みを取り消す・やり直すボタンがあります (7.4 節)。

ダンプ

RAM の中身を 1 行に 8 バイトずつ、番地、+0〜+7 のバイト、Σ (チェックサム) の順に並べます (7.3 節)。実行を始めてから書き換わったバイトは色を変えます。

逆アセンブル

カーソルのある命令のまわりを、命令ごとに番地、バイト、ニーモニックの順に出します。データとみたところは DB の行にして、色を変えて出します (7.6 節)。行の右端の印は、命令かデータかをどう見分けたかを表します。行の左の丸をクリックすると、その命令にブレークポイントを設定します (もう一度クリックすると解除します)。

レジスタ

I、J、A、B、K、L、M、N、X、Y、DP、PC、P、Q、R、キャリー (C) とゼロ (Z) のフラグ、実行を始めてからのサイクル数です。前に止まったときから変わったレジスタは色を変えます。

内部 RAM

CPU の内部 RAM(00〜5F 番地) を 16 バイトずつ出します。見出しの「内部 RAM」をクリックすると、開く・閉じるを切り替えます。

知らせ

いちばん下に、止まった理由や、操作の結果が出ます。

7.2　BASIC の CALL で呼んで調べる

command / Ctrl + D を押すか、操作欄の「作業欄 ▶」のボタンか「表示・入力」のメニューの「マシン語の作業欄を表示」を選ぶと、本体の右に作業欄が表示されます。もう一度 command / Ctrl + D を押すか「◀ 作業欄」で隠せます。

本体は、作業欄を出しても BASIC で動き続けます。作業欄の見出しの横に「キーは本体へ」と出ているあいだは、パソコンのキーは本体に届きます。作業欄をクリックすると「キーは作業欄へ」に変わり、ダンプの打ち込みと作業欄のショートカット (command / Ctrl + E、command / Ctrl + V なども)

が作業欄に届きます。Esc を押すと、キーは本体に戻ります。

作業欄で打ち込んだマシン語は、BASIC の CALL で呼びます。開始番地の横に、CALL &C000 のように打つ命令が出ます。開始番地の決め方は 7.5 節にあります。BASIC が次の行を打つキーを待っているときに「▶ 実行」(command/Ctrl+G) を押すと、CALL &C000 のように開始番地を呼ぶ行を打って ENTER を押したのと同じになり、マシン語が RTN で戻ると BASIC に戻ります。液晶は消しません。ROM なしの BASIC が動いているあいだ (マシン語を実行していないとき) は、1 命令ずつは進められません。

ブレークポイントを設定しておくと、CALL で呼んだマシン語がその命令に来たときに本体の実行が一時停止し (command/Ctrl+P で止めたときと同じ状態)、作業欄を隠していれば作業欄が表示されます。そこから「ステップ」や「CALL 単位で実行」で進め、「▶ 続行」で本体の実行を続けます。ROM 使用時は、ROM の中の命令にもブレークポイントを設定でき、ステップ実行できます。

7.3　ダンプリストの読み方

本や雑誌のマシン語のプログラムは、ダンプリストの形で載っていることがよくあります。ダンプリストの 1 行は、番地、バイトを 16 進表記で並べたもの、チェックサムでできています。

```
C000 027F10F8005237 12
```

番地の C000 は、その行の最初のバイトを置く番地です。続く 027F10F8005237 は 02、7F、10、F8、00、52、37 の 7 バイトで、C000〜C006 番地に置きます。長いプログラムでは 1 行に 8 バイトずつ並び、行の番地は 8 ずつ増えます。最後の 12 がチェックサムで、行のバイトをすべて足した和 (212) の下 2 桁です。打ち込んだ行のチェックサムをリストのチェックサムと比べると、打ち間違えた行を見つけられます。チェックサムの計算のしかたは、本や雑誌によって違うことがあります。

例の 7 バイトは、PC-1251 の液晶の左端の 1 列に縦の線を描いて戻るマシン語です (LIA 7F、LIDP F800、STD、RTN)。ROM のルーチンを呼ばないので、ROM なしでも打ち込んで実行できます。

作業欄のダンプも 1 行に 8 バイトを並べ、右端の Σ の列に、その行の 8 バイトの和の下 2 桁を出します。ダンプの行は開始番地から 8 バイトずつ区切るので、ダンプリストの最初の番地から打ち込めば、行の区切りもダンプリストとそろいます。打ち込んだ行の Σ がリストのチェックサムと違えば、その行に打ち間違いがあります。

7.4　ダンプを打ち込む

編集中は、パソコンのキーで作業欄のダンプに書き込みます。ダンプリストは、次の順に打ちます。

1. 番地の 4 桁を打ちます。打ち終えると、カーソルがその番地の行のバイトの欄へ移ります。開始番地がまだ決まっていなければ、開始番地もその番地になります。カーソルがバイトの欄にあるとき (別の範囲を打つときなど) は、先に Tab を押して番地の欄へ移ります。起動したときと、保存した状態に戻したときは、カーソルは番地の欄にあります。

2. バイトを 2 桁ずつ打ちます。番地のすぐあとと、バイトとバイトのあいだの空白は、打っても読み飛ばします。1 行の 8 バイトを打ち終えたら、Σがリストのチェックサムと同じか確かめます。チェックサムは打ちません。
3. Enter で次の行の最初のバイトへ移ります。行の番地は自動で 8 つ先になるので、2 行目からはバイトだけを打ちます。

キー	すること
0 〜 9、A 〜 F	カーソルの所に 16 進表記の 1 字を書く
Enter	次の行の最初のバイトへ移る
Space	番地の欄からバイトの欄へ移る。バイトの欄では次のバイトへ移る。番地の 4 桁やバイトの 2 桁を打った直後は、区切りとして読み飛ばす
Tab	番地の欄の最初の桁へ移る。番地を 4 桁打つと、その番地の行へ移る (メモリは書き換えない)
← →	1 字ずつ戻る・進む。行の最後のバイトから進むと次の行の最初のバイトへ、行の最初のバイトから戻ると前の行の最後のバイトへ移る (Backspace も 1 字戻るだけで、字は消さない)
↑ ↓	1 行上・下へ移る
PageUp PageDown	1 画面ぶん上・下へ移る
Home End	行の最初のバイト・行の終わりへ移る (command / Ctrl + ← ・ command / Ctrl + → でも同じ)。行の最初のバイトで Home を押すと番地の欄へ移る
command / Ctrl + ↑ command / Ctrl + ↓	開始番地の行・書いてあるところの終わりへ移る
command / Ctrl + Z	直前の書き込み (1 バイトの打ち込みか、1 回の貼り付け) を取り消す
shift + command / Ctrl + Z	取り消した書き込みをやり直す

マウスでは、ダンプの番地やバイトをクリックするとカーソルがそこへ移り、作業欄の上でホイールを回すとダンプが 1 行ずつ動きます。

書き込みの取り消しとやり直しは、「取り消す」「やり直す」のボタンと、操作欄の「プログラム」のメニューからもできます。取り消せるのは作業欄で書き込んだもの (打ち込みと貼り付け) だけで、マシン語を実行して書き換わったメモリは戻りません。

■貼り付ける　ダンプリストや.hex の行をクリップボードにコピーして command / Ctrl + V を押すと、まとめて書き込みます。チェックサムは、バイトのあとに空白で区切っても (`C000 027F10F800523700 12`)、`:` で区切っても (`C000 027F10F800523700:12`) かまいません。チェックサムのついた行は、バイトの和とチェックサムを比べ、合わない行の番地を下の知らせに出します。

7.5　実行して調べる

すること	ボタン	キー
開始番地から実行する	▶ 実行	command / Ctrl + G
カーソルの番地を開始番地にする		command / Ctrl + A
カーソルの番地を開始番地にして実行する		shift + command / Ctrl + G
実行を一時停止する	■ 一時停止	command / Ctrl + .（command / Ctrl + P でも同じ）
一時停止したところから続行する	▶ 続行	command / Ctrl + G
次の 1 命令だけ実行して止まる	ステップ実行、ステップ	command / Ctrl + J
呼び出しの命令を、呼んだ先から戻るまで実行して止まる	CALL 単位で実行	shift + command / Ctrl + J
カーソルの命令にブレークポイントを設定・解除する	● ブレークポイント	command / Ctrl + B
一時停止中に、実行をやめて編集に戻る	編集へ	Esc

■開始番地を決める　開始番地は、まだ決めていなければ、ダンプに最初に打った番地や、読み込んだプログラムの CALL の番地になります。自分で決めるときは、次のどれかで決めます。

- ダンプのカーソルを置いて command / Ctrl + A を押す。
- ダンプのバイトか逆アセンブルの行を右クリックし、「C010 番地を開始番地にする」のような項目を選ぶ。
- 「開始番地」の欄をクリックし、16 進表記の 4 桁を打つ。Esc でやめます。

マシン語が最後に RTN で戻ると実行が終わり、編集中に戻ります。BASIC から CALL で呼ぶマシン語と同じく、最後は RTN で戻るように書いてください。

実行中は、パソコンのキーが本体のキーになります（Tab が SHIFT、option（Windows と Linux では Alt）が DEF、Delete が CL、Esc が BRK）。キーを読むマシン語も、そのまま試せます。裏の RESET ボタン（command / Ctrl + R、F5）を押すと、実行をやめて編集中に戻ります。

編集中に command / Ctrl + J を押すと、開始番地の最初の命令を実行する前で止まります。そこから command / Ctrl + J を押すたびに、1 命令ずつ進みます。次の命令が CAL や CALL のような呼び出しのときは、「CALL ごと」（shift + command / Ctrl + J）で、呼んだ先が RTN で戻ってくるまで実行し、呼び出しの次の命令で止まれます。

ブレークポイントは、逆アセンブルの行の左の丸をクリックするか、ダンプのカーソルを命令に置いて command / Ctrl + B を押すか、ダンプや逆アセンブルの行を右クリックして置きます。実行がその命令に来ると、命令を実行する前で止まります。

止まったときは、ダンプのカーソルが次に実行する命令へ移り、変わったレジスタやメモリの値は色が変わります。

マシン語が RAM の外の番地 (ROM の中のルーチンなど) へ飛ぶと、飛んだ先で止まり、下の知らせに番地と、わかれば ROM のルーチンの名前を出します。ROM なしではその先を実行できないので、「編集へ」で編集に戻ります。

7.6　命令とデータを見分ける

マシン語のプログラムには、命令のあいだに文字や表や字の形などのデータが入っています。作業欄の逆アセンブルは、命令とデータを見分けて、データは DB の行にして出します (`DB 48 45 4C 4C 'HELL'`)。行の右端には、どう見分けたかの印が出ます。

印	見分け方
✓	実行済み。作業欄を出しているあいだに、CPU がその番地の命令を実行した
.	静的解析。開始番地などの入口から、分岐と呼び出しをたどって命令と判定した
?	推定。どこからも参照されていないので、中身の形から推定した
U	ユーザ指定。右クリックで、データか命令に指定した (下の「ユーザ指定」)

PTC と DTC の表は、表の 1 行ずつを DB の行にして、行き先を添えます (`DB 01,C1,23 ; → C123`)。ROM 使用時は、マシン語を CALL で呼ぶ前に作業欄を出しておくと、実行した命令に ✓ の印がつきます。

■ユーザ指定　見分けが違っているときは、ダンプのバイトか逆アセンブルの行を右クリックして、次の項目を選びます。

C010 番地からデータにする

その番地から、次の指定か実行済みの命令の手前までをデータに指定します。

C010 番地から命令にする

その番地を命令の頭に指定し、そこから静的解析します。

C010 番地の指定を消す

指定をやめて、推定に戻します。

.hex に書き出すと、データの指定は`# data:` の行に入ります (7.7 節)。

7.7　打ち込んだマシン語をファイルに書き出す

作業欄で打ち込んだマシン語は、.hex のファイルに書き出して残せます。command/Ctrl+E か「.hex に書き出す」のボタンを押すと、プログラムのフォルダの機種のフォルダ (`~/.sc61box/programs/pc1251` など) に、`dump-日時.hex` の名前で書き出します。書き出すのは、作業欄から書いた番地 (打ち込み、貼り付け、一覧からの読み込みで書いたところ) です。書き出す範囲とバイト数は、「.hex に書き出す」のボタンの下に出ます。

範囲は自分で決めることもできます。範囲の行をクリックし、始めと終わりの番地を 16 進表記で 4 桁ずつ打ちます (`C000C0FF` と打てば `C000`〜`C0FF` 番地。あいだに空白や-を打ってもかまいません)。終わりの番地も範囲に入ります。`C000-C0FF` のような文字を command/Ctrl+V で貼り付けて

もかまいません。範囲の行には「指定」と出て、右端に「自動に戻す」のボタンが出ます。このボタンを押すか、範囲の行をクリックして何も打たずに Enter を押すと、書いた番地から決める範囲に戻ります。Esc で打つのをやめます。BASIC の POKE で書いたマシン語のように、作業欄から書いていないところも、範囲を決めれば書き出せます。プログラムを読み込むと、決めた範囲は消えます。

書き出したファイルの各行には、番地とバイトのあとに、; に続けてニーモニックを書き添えます。頭には# run: CALL &開始番地の行が入ります。

```
; C000〜C006(マシン語の作業欄から書き出し)
# run: CALL &C000
C000 02 7F       ; LIA 7F
C002 10 F8 00    ; LIDP F800
C005 52          ; STD
C006 37          ; RTN
```

shift+command / Ctrl+E か「ダンプリスト」のボタンを押すと、本や雑誌のダンプリストの形 (番地、8 バイト、:、チェックサム) で dumplist-日時.hex に書き出します。行は書き出す範囲の始めから 8 バイトずつ区切ります。このファイルも一覧から読み込め、行をコピーして貼り付けることもできます。

```
; C000〜C006(マシン語の作業欄から書き出し)
# run: CALL &C000
C000 027F10F8005237:12
```

8 PC-1250・PC-1255 として使う

PC-1250 と PC-1255 は、PC-1251 と同じ ROM で、違うのは RAM の範囲だけです。起動のときに--model 1250 か--model 1255 を付けると、その機種として動きます。ROM は PC-1251 の cpu-1251.rom と bas-1251.rom を使います。

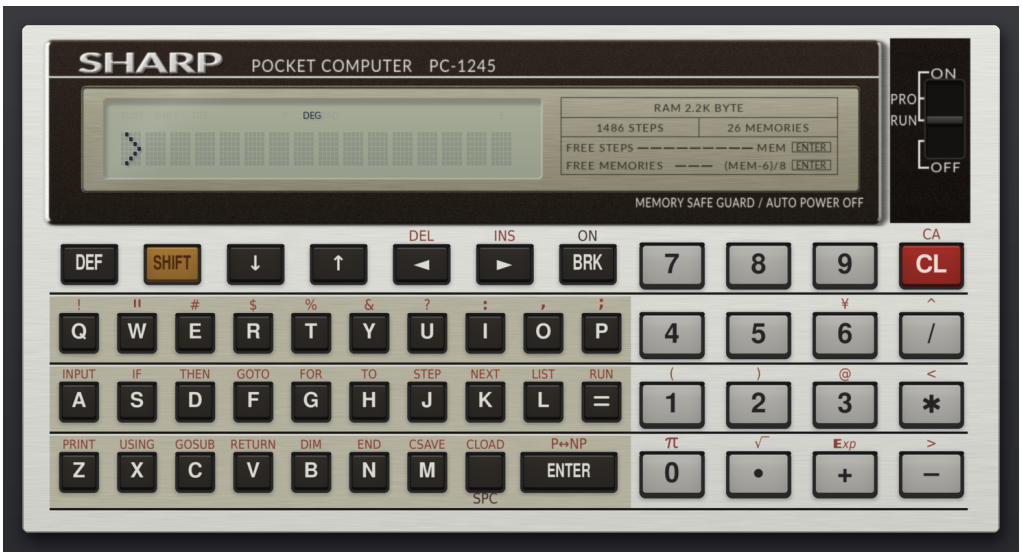
機種	RAM	BASIC のプログラムの先頭	使える大きさ (MEM)
PC-1250	C000〜C7FF(2KB)	C030	1438 バイト
PC-1251	B800〜C7FF(4KB)	B830	3486 バイト
PC-1255	A000〜C7FF(10KB)	A030	9630 バイト

プログラムの一覧には、PC-1251 用のもののうち、マシン語が RAM に収まり、BASIC のプログラムの領域にかからないものが出ます。PC-1250 では、C000 番地から置くブロック崩しなどは出ません。

9　PC-1245 として使う

PC-1245 は、PC-1251 とは別の ROM を使う機種です。起動のときに`--model 1245` を付けると、PC-1245 として動きます。

```
$ uv run sc61box --model 1245
```



9.1　液晶

液晶は 16 桁です。PC-1251 と同じ液晶 RAM のうち、左の 12 桁 (`F800`〜`F83B`) と、右側の初めの 4 桁 (`F87B`〜`F868`) だけが見えます。17 桁目より右に書いたものは表示されないので、PC-1251 用のプログラムには表示が途中で切れるものがあります。

表示記号は `BUSY`、`DEF`、`SHIFT`、`P`、`DE`・`G`・`RAD`(`DEG` などの角度の単位)、`E` です。モード (`PRO`、`RUN`) は液晶に出ず、スイッチの位置で見ます。表示記号の位置は、`DEG` のほかは推定です。

9.2　モードスイッチ

スイッチは PRO、RUN、OFF の 3 段です。PRO と RUN の 2 段は「ON」の括りに入っていて、電源が入る位置です。RSV モードはないので、4.6 節の文字列の登録は使えません。

9.3　キー

キーの位置は PC-1251 と同じですが、刷ってある文字が違います。

- A〜=の段とZ〜SPCの段は、SHIFTを押してから打つと、キーの上に刷ってある BASIC の命令が入ります。たとえばSHIFT Zで PRINT、SHIFT Gで FOR、SHIFT =で RUN、SHIFT SPCで CLOAD です。
- 括弧はSHIFT 1が (、SHIFT 2が) です (PC-1251 は↓と↑)。パソコンで(、)を打つと、SHIFT 1、SHIFT 2に直して打ちます。
- ほかの記号は付録 B の表と同じです。

9.4　メモリと見本のプログラム

RAM は 2KB(C000〜C7FF) で、BASIC のプログラムは C000 番地から置かれます。プログラムに使えるのは 1486 ステップ (バイト) です。

見本のプログラムは同じ一覧から読み込めますが、次の点に気をつけてください。

- ブロック崩し (5.6 節) は、programs/pc1245 にある PC-1245 用の break.hex と break.bas が読み込まれます。遊ぶ場所は 75 列で、得点は終わってから出します。点滅するブロックはなく、玉はいつも 1 個です。
- ほかの見本は動きますが、24 桁を使う表示は 17 桁目から右が見えません。

10　PC-1261 として使う

PC-1261 は、2 行 ×24 桁の液晶を持つ機種です。ROM は PC-1251 とは別のもので、BASIC の命令も少し違います。起動のときに--model 1261 を付けると、PC-1261 として動きます。

```
$ uv run sc61box --model 1261
```



PC-1260 と PC-1262 は PC-1261 と同じ ROM で、違うのは RAM の大きさだけです (--model

1260、--model 1262)。

10.1　液晶

液晶は 2 行 ×24 桁です。液晶の下の BUSY、PRINT、DEG、RAD、ERROR の字のすぐ上に、液晶の短い横棒が点きます。ERROR の右には字のない横棒がもう 1 つあり、207C 番地のビット 6 で点きます (RUN モードで POKE &207C,&40 と打つと点き、POKE &207C,0 か電源を入れ直すと消えます。ROM 自身は点けません)。液晶の左端には、上からカナ、SMALL、SHIFT、DEF の記号が並んでいます。SMALL は英小文字の入力の記号で、RUN モードで POKE &203C,PEEK &203C AND &7F と打つと、SHIFT 8 で英小文字の入力に切り替わるようになります。

10.2　キー

キーの並びは PC-1251 と同じで、刷ってある文字が違います。

- 括弧は SHIFT 1 が (、SHIFT 2 が)、¥は SHIFT 6 です。パソコンで (、)、¥ を打つと、エミュレータがそのキー操作に直して打ちます。
- Exp のキーはありません。指数は英字の E で書きます (1E3 など)。
- SHIFT 7 (HELP) で、BASIC の命令の一覧が出ます。
- SHIFT 8 (カナ) で、カナの入力に切り替わります。もう一度押すと英字の入力に戻ります。カナの入力のあいだは、各キーの右下に刷ってあるカナが入り、SHIFT を押してから打つと右上のカナが入ります。「・」の印のあるキーは、SHIFT を押してから打つと小さい字 (ァ、ッ、ャなど) になります。数字の段の右に刷ってある゛゜。、「」・ーも、カナの入力のときに打てます。

パソコンでカナ (全角のカタカナ、半角のカタカナ、ひらがな) を打つか貼り付けると、エミュレータがカナの入力に切り替えて打ち、打ち終わったら英字の入力に戻します。「ガ」のような濁点のついた字は、「カ」のあとに゛を打つ形に直します。ファイルの読み込みでも同じです。

貼り付けは、1 打鍵に 0.22 秒ほどかかります。

10.3　メモリと見本のプログラム

RAM は 10KB(4000〜67FF) で、BASIC のプログラムは 4080 番地から置かれます。プログラムに使えるのは 9342 バイト (MEM の値) です。PC-1260 は RAM が 4KB(5800〜67FF) で、使えるのは 3198 バイトです。

PC-1261 用の見本は programs/pc1261 にあります。ball.hex と ball.bas は、ボールが 2 行の液晶いっぱいに弾みながら右へ進むマシン語です (CALL &5A00 で動かし、Esc (BRK) で BASIC に戻ります)。hello.bas は CURSOR で表示の位置を決めて、2 行の液晶にカナと記号を出します。PC-1260 は RAM が狭いので、コンパイラ LC-3 とその見本は使えません。

■コンパイラ LC-3　programs/pc1261/lc3.hex は、COR 氏のコンパイラ LC-3(ポケコン・ジャーナル 1989 年 6 月号) です。1 バイトの整数だけを扱うゲーム向けのコンパイラで、BASIC

のプログラムとして打ち込んだソースをマシン語に直して動かします。一覧の「LC-3 コンパイラ」を選ぶと、BASIC のプログラムの領域を動かしてコンパイラを書き込みます。そのあと PRO モードでソースを打ち込み、`CALL &4000` でコンパイル、`CALL &4900` で実行します。記事の見本のうち 3 本 (`lc3-hello`、`lc3-move`、`lc3-chars`) と、ブロック崩し (`lc3-block`。パドルは 8 と 2 で動かす) は、選ぶとコンパイルから実行まで続けて行います。コンパイラも見本も、ROM なしで動きます (6 節)。コンパイラは https://github.com/hiroog/LC3_Compiler の `LC-3.bin` を元にしました。コンパイラと記事の見本のソースは COR 氏 (GitHub では hiroog 氏) の作品で、エミュレータの MIT ライセンスには含まれず、作者のブログの記事 (https://wlog.flatlib.jp/archive/1/2018-1-3) の、自由に配布してよいという言葉に従って同梱しています。

11　PC-1350 として使う

PC-1350 は、150×32 ドットの液晶 (4 行 ×24 桁の字) を持つ機種です。ROM は PC-1251 とは別のもので、モードスイッチもありません。起動のときに`--model 1350` を付けると、PC-1350 として動きます。

```
$ uv run sc61box --model 1350
$ uv run sc61box --model 1350 --card 16
```



ROM は、カナの打てる国内向けのもので確かめました。`--overseas` をつけると輸出向けの配線で動き、カナの代わりに英小文字が入ります。

11.1　電源とモード

モードスイッチはありません。RUN モードと PRO モードは MODE キーで切り替えます。パソコンでは F1 で RUN モード、F2 で PRO モードにします (いまのモードと違えば MODE キーを押します)。いまのモードは、液晶の左端の `RUN・PRO` の表示記号で見ます。

本体の右上のスイッチは電源の ON・OFF です。マウスでクリックするかドラッグするか、F3(ON)・F4(OFF) で動かします。電源を切ってもプログラムは残ります。スイッチが OFF のあいだは、ONキー (Esc) では電源が入りません。

11.2　液晶

液晶は 150×32 ドットで、字は 4 行 ×24 桁です。液晶の左端に、RUN、PRO、カナ、SML、SHIFT、DEF の表示記号が縦に並びます。

11.3　キー

キーは PC-1350 の配置です。

- 括弧、/、:、;、, は数字の側のキーで、SHIFTを押さずに入ります。<、>、¥、^は、数字の側のキーをSHIFTを押してから打ちます。パソコンで記号を打つと、エミュレータがそのキー操作に直して打ちます。
- Exp のキーはありません。指数は英字の E で書きます (1E3 など)。
- パソコンのDeleteはDEL、InsertはINS、HomeはCLSです。Backspaceは◀とDELで 1 字消します。
- カナキー (パソコンではF8) でカナの入力に、SHIFTカナで英小文字の入力 (SML) に切り替わります。パソコンで打つか貼り付けたカナは、PC-1261 と同じようにカナの入力に切り替えて打ちます (10 節)。
- ファイルの BASIC の英小文字は、「"」で囲んだ文字列の中だけ SML に切り替えてそのまま打ち、ほかは大文字にします。命令は大文字でないと通りません。

貼り付けは、1 打鍵に 0.25 秒ほどかかります。

11.4　グラフィック

液晶の点は PSET・PRESET・LINE・GPRINT で描き、POINT で読みます。座標は左上が (0,0)、右下が (149,31) です。プログラムの中の描く命令は、PRINT と同じように、WAIT を実行していなければENTERを待ちます。描いた点は CLS で消え、PRINT やENTERで字を書き直した行でも消えます。

11.5　シリアル入出力

エミュレータには、シリアルの端子につなぐ相手がありません。OPEN、CLOSE、CONSOLE は実機と同じように設定を変え、OPEN$でいまの設定を読めます (初めは 1200,N,8,1,A,C,&1A で、電源を入れると初めの設定に戻ります)。OPEN したシリアルは、RUN とプログラムの終わりでも閉じます (設定はそのまま)。OPEN したあとの SAVE、LOAD、LLIST と、プログラムの中の LPRINT は、相手を待って止まるので、BRKで止めます。何もつないでいない実機と同じ動きです。CLOSE すると、LLIST と LPRINT はプリンタに戻り、SAVE と LOAD は何もしなくなります。

11.6　TEXT モードと BASIC モード

PRO モードで TEXT と打つと、プログラムの命令を字の並びに直してテキストモードになり、プロンプトが<になります。テキストモードで打った行は、命令に直さず、打ったとおりの字で入ります。BASIC と打つと、行を命令に直して BASIC モードに戻ります。電源を入れると BASIC モードに戻りますが、プログラムは字の並びのまま残ります。

11.7　メモリと見本のプログラム

RAM は本体の 4KB で、プログラムに使えるのは 3070 バイト (MEM の値) です。起動のときに--card 8 か--card 16 をつけると、8KB か 16KB の RAM カード (CE-202M) を差した状態になり、使える大きさは 11262 バイトか 19454 バイトになります。

PC-1350 用の見本は programs/pc1350 にあります。hello.bas は CURSOR で表示の位置を決めて、4 行の液晶にカナと英小文字を出します。ball.hex と ball.bas は、ボールが液晶のいちばん下で弾んでいちばん上まで上がりながら右へ進むマシン語で、地面に着くたびにつぶれて低い音が鳴ります (CALL &6100 で動かし、Esc(BRK) で BASIC に戻ります)。グラフィックの命令の見本は 2 つです。lines.bas は、端が液晶の縁で跳ね返りながら動く線を、LINE の,X で描いて 10 本前の線を消します。snake.bas はヘビのゲームで、8 2 4 6(ゲームパッドは十字キー) で向きを変えてえさを食べます。ヘビの体は消えずに伸び続け、壁か体にぶつかると終わります。

付録 A　起動のオプション

オプション	意味
プログラム (名前)	読み込むプログラム (ファイルの場所か、プログラムのフォルダにある名前)
--run	読み込んだあと実行する
--scale 数	拡大率 (既定は 1.0。画面に収まらなければ縮める)
--fresh	保存した RAM を読まず、終わるときにも保存しない
--mute	音を出さない
--persist 数	液晶の残像の強さ (既定は 1.0、0 で残像なし)
--type ファイル	起動後にファイルの中身をキーとして打ち込む
--load ファイル	起動後に 16 進ダンプを RAM へ書き込む
--export フォルダ	保存した RAM の BASIC のプログラムを.bas で書き出して終わる
--model 機種	1251(既定)、1250、1255、1245、1260、1261、1262、1350 のどれか。PC-1250・PC-1255 は 8 節、PC-1245 は 9 節、PC-1261 は 10 節、PC-1350 は 11 節
--card 大きさ	PC-1350 に差す RAM カードの大きさ (KB)。0(既定)、8、16
--overseas	PC-1350 を輸出向けの配線で動かす (カナの代わりに英小文字が入る)
--numpad	矢印キーを数字キーの 8・2・4・6 として押す状態で始める (command/Ctrl+K で切り替え)
--lcd	本体の画像を出さず、液晶だけを見せる

オプション	意味
`--tape` ファイル	カセットの音 (wav) をセットして始める (5.11 節)
`--state` ファイル	保存した状態から始める (3.6 節)
`--norom`	ROM を使わず、ROM なしの BASIC で動かす (6 節)。ROM が見つからなければ、つけなくても ROM なしで起動する
`--no-basic`	ROM なしのとき、BASIC を付けずにマシン語の作業欄だけで動かす
`--new-routine` 番地	代替 ROM ルーチンを書き始めるファイルを作って終わる (6.2 節)。番地: 名前とも書ける
`--check-routines` 対象	代替 ROM ルーチンを ROM 使用時の動きと比べて終わる (6.2 節)。対象は `user`、`all` か名前
`--list-routines`	代替 ROM ルーチンの一覧を出して終わる (6.2 節)
`--help`	オプションの一覧を出す
`--version`	版を出して終わる

環境変数	意味
`SC61BOX_ROM`	ROM を置いたフォルダ
`SC61BOX_PROGRAMS`	プログラムのフォルダとして先に探すフォルダ。いくつも書くときは、macOS では:、Windows では; で区切る
`SC61BOX_LANG`	画面の言語。`ja`(日本語) か `en`(英語)。`--lang` があれば`--lang` のほうを使う

付録 B　SHIFT で入る記号

パソコンで次の記号を打つと、右のキー操作に直して打ち込みます。

記号	キー	記号	キー	記号	キー
!	SHIFT+Q	&	SHIFT+Y	(	SHIFT+ ↓
"	SHIFT+W	?	SHIFT+U	)	SHIFT+ ↑
#	SHIFT+E	:	SHIFT+I	@	SHIFT+3
$	SHIFT+R	,	SHIFT+O	<	SHIFT+*
%	SHIFT+T	;	SHIFT+P	>	SHIFT+-
^	SHIFT+/	√	SHIFT+.	π	SHIFT+0
€	SHIFT++	¥	SHIFT+6		

PC-1245 では (と) が SHIFT+1 と SHIFT+2 になります (9 節)。表の€ は、指数を表す E(`Exp`) の代わりに使う字で、`[E]` と書いてもかまいません (PC-1261 と PC-1350 では英字の `E` として打ちます)。\は¥として打ちます。

SHIFT + は指数の E(`Exp`)、SHIFT ◀ は 1 字消す (`DEL`)、SHIFT ▶ は 1 字あける (`INS`)、SHIFT ENTER は P ↔ NP(プリンタを使うかどうかの切り替え)、SHIFT CL は `CA`(すべて消す) です。

付録 C　メモリマップと入出力

番地	中身
0000–1FFF	CPU の内部 ROM(8KB)
4000–7FFF	BASIC ROM(16KB)
B800–C7FF	RAM(4KB)。B000–B7FF は B800–BFFF と同じものが見える。PC-1245 は C000–C7FF(2KB)
F800–F87F	液晶の RAM。F800–F8FF と同じものが F900–FFFF の 256 バイトごとに見える。PC-1245 は E800–EFFF にも見える

- CPU は SC61860 で、命令サイクルは 192kHz(クロック 576kHz の 1/3) です。資料にない命令 72h(LIIH) は 2 バイトの命令、54h(READM) は 56h(READ) と同じ形の 1 バイトの命令として扱います。
- モードスイッチは、ポート IB のビット 3 を出して INB 命令で読みます。ビット 0 が RSV、ビット 1 が PRO、ビット 2 が OFF で、どれも立っていなければ RUN です。
- C ポートのビット 2 を立てると、CPU はキーが押されるか 512ms のタイマが変わるまで止まって待ちます。ROM はキーを待つあいだ、C ポートのビット 2 を立てて止まり、起きた回数を数えてオートパワーオフの時間を計ります。
- ブザーは C ポートのビット 4〜6 で鳴らします。BASIC の BEEP は内蔵の 4kHz の発振を使い、マシン語ではビットを直接切り替えて好きな高さの音を出せます。
- 液晶は、左半分の 60 列が F800〜F83B の順に、右半分が F87B から F840 へ逆の順に並びます。1 バイトが縦 7 ドットの 1 列で、ビット 0 が上です。表示記号は F83C〜F83E のビットです。

PC-1250・PC-1255

RAM は PC-1250 が C000〜C7FF、PC-1255 が A000〜C7FF です。ほかは PC-1251 と同じです (8 節)。

PC-1261

番地	中身
0000–1FFF	CPU の内部 ROM(8KB)
2000–20FF、2800–28FF	液晶の RAM。2100–27FF には 2000–20FF、2900–2FFF には 2800–28FF と同じものが見える
4000–67FF	RAM(10KB)。PC-1260 は 5800–67FF(4KB)
8000–FFFF	BASIC ROM(32KB)

- 液晶の RAM は、どちらも 00〜3B が 1 行目、40〜7B が 2 行目で、2000 の側が左の 12 桁、2800 の側が右の 12 桁です。表示記号は 203D と 207C のビットです。2880〜28FF は表示に

使わず、ROM が式の計算の途中の値を積むのに使います。

- 何もつながっていない番地を読むと、番地の上位バイトが返ります。
- CPU のクロックは 768kHz です。キーの並び、モードスイッチ、タイマ、ブザー、カセットのポートは PC-1251 と同じです。

PC-1350

番地	中身
0000-1FFF	CPU の内部 ROM(8KB)
2000-5FFF	RAM カード。8KB のカードは 4000-5FFF(2000-3FFF にも同じものが見える)、16KB のカードは 2000-5FFF
6000-6FFF	本体の RAM(4KB)
7000-79FF	液晶の RAM。ROM は液晶の RAM の使わないところと 7A00-7DFF を作業域に使う
7E00-7FFF	キーのポート
8000-FFFF	BASIC ROM(32KB)

- 液晶の RAM は横 30 列ずつの 5 つの区画 (7000、7200、7400、7600、7800) に分かれ、各区画は上から 8 ドットずつの 4 段です。段の頭は、区画の頭から +00、+40、+1E、+5E です。1 バイトが縦 8 ドットで、ビット 0 が上です。表示記号は 783C のビットで、ビット 0 が SHIFT、1 が DEF、4 が RUN、5 が PRO、6 がカナ、7 が SML です。
- キーは 2 つの組に分かれます。1 つは 7E00〜7FFF に書いた値のビット 0〜6 を出して INA で読む組、もう 1 つは IA のビットを 1 本出して INA でほかのビットを読む組です。ON キーは TEST 08 で読みます。
- CPU のクロックは 768kHz です。

付録 D　実機と違うところ

- 実機と並べて動きを比べてはいません。
- ROM なしの BASIC と ROM 使用時の違いは、6.1 節にまとめました。
- カセットテープの入出力は、実機用のテープを録音した wav を CLOAD で読み込めるところまで確かめました。エミュレータが書き出した wav を実機で読み込ませることは、まだ確かめていません (5.11 節)。
- プリンタの紙送りのキーは再現していません (5.10 節)。
- プリンタは送られたバイトをすぐに受け取るので、プログラムは印字を待たずに先へ進みます。
- ROM の版の違いは確かめていません。
- 内部 ROM が BASIC の PEEK でも読めます。資料では、実機の内部 ROM は DATA(35h) 命令でしか読めないとされています。

付録 E　ファイルの一覧

場所	中身
`sc61box/`	エミュレータ本体 (CPU、メモリとポート、画面、音、プログラムの読み込み)
`programs/`	見本のプログラム (機種ごとのものは `pc1251/`、`pc1245/`、`pc1261/`、`pc1350/`)
`rom/`	自分で用意した ROM(2.4 節、PC-1245 は 9 節)
`tests/`	動作の試験 (`uv run pytest -q tests`)
`doc/`	この手引きと『PC-インタープリタを読む』(`pcinterp.pdf`)
`~/.sc61box/`	保存した RAM、画面の画像、プログラムのフォルダ

付録 F　謝辞

このエミュレータは、次のエミュレータの実装を参考にして作りました。作者の方々にお礼申し上げます。

- MAME(https://github.com/mamedev/mame)
- PockEmul(https://github.com/paulie-g/PockEmul)
- PokecomGO(https://github.com/digihori/pokecom)
- Pokecom GO Plus(https://github.com/digihori/pokecom-go-plus)

このエミュレータの作成には Anthropic の Claude を利用しています。